

ANOVA

v1.0.0

Allan Omondi

20 June 2026

Table of contents

1	Generate the Synthetic Dataset	3
1.1	4 Algorithms, 10 independent datasets, and 5 independent runs	3
1.2	NOT the same as Cross-Validation with Repeats	4
1.3	Valid ways to Create Independent Datasets	4
2	Load the Dataset	5
3	Initial EDA	5
3.1	<u>Measures of Frequency</u>	7
3.2	<u>Measures of Central Tendency</u>	7
3.3	<u>Measures of Distribution</u>	9
3.3.1	Variance	9
3.3.2	Standard Deviation	9
3.3.3	Kurtosis (Pearson)	10
3.3.4	Skewness	10
3.4	<u>Basic Visualizations</u>	11
3.4.1	Histogram	11
3.4.2	Box and Whisker Plot	15
3.4.3	Missing Data Plot	20
3.4.4	Scatter Plot	22
4	Statistical Test: ANOVA	26
4.1	One-Way ANOVA	26
4.2	Two-Way ANOVA	28
4.3	Multiple Analysis of Variance (MANOVA)	29
5	Diagnostic EDA (Model Diagnostic)	33
5.1	<u>Test of Linearity — Not Applicable</u>	33

5.2	<u>Test of Independence of Errors (Autocorrelation)</u>	33
5.3	<u>Test of Normality of the Distribution of the Errors</u>	35
5.4	<u>Test of Homoscedasticity (Homogeneity of Variance)</u>	40
5.5	<u>Quantitative Validation of Assumptions — Not Applicable</u>	42
6	Interpretation of the Results	42
6.1	One-Way Analysis of Variance	44
6.1.1	Technical Academic Report (Based on APA Guidelines of Reporting Results)	44
6.1.2	Non-Technical Business Report	46
6.1.3	Recommendations	46
6.2	Two-Way Analysis of Variance	47
6.2.1	Technical Academic Report (Based on APA Guidelines of Reporting Results)	47
6.2.2	Non-Technical Business Report	48
6.2.3	Recommendations	49
6.3	Multiple Analysis of Variance	49
6.3.1	Technical Academic Report (Based on APA Guidelines of Reporting Results)	49
6.3.2	Non-Technical Business Report	51
6.3.3	Recommendations	51
7	References and Further Reading	52

```
# `installed.packages()` retrieves a matrix of all installed packages
# `[ , "Package"]` extracts on the "Package" column from the matrix of all
# packages. This contains the name of the package.
# The %in% operator is used to test if the specified package is in the matrix of
# all packages
# `dependencies = TRUE` instructs R to install not only the specified package but
# also its dependencies

# `pacman::p_load("here")` installs the package called "here". This package is used
# in the next line.
# `knitr::opts_knit$set(root.dir = here::here())` is used to ensure that the "knitr"
# utility in R knows where to find the files required to create the HTML, Word, or
# PDF version of the notebook.

if (!"pacman" %in% installed.packages()[ , "Package"]) {
  install.packages("pacman", dependencies = TRUE)
  library("pacman")
}
```

```
# For reading data from a CSV file
pacman::p_load("readr")
# For data visualization
pacman::p_load("ggplot2")
# For data visualization of missing data
pacman::p_load("naniar")
# "Companion to Applied Regression" for diagnostic EDA using
pacman::p_load("car")
# For the Durbin-Watson Test (test of independence of errors [autocorrelation])
pacman::p_load("lmtest")
# For performing a test of normality for MANOVA
pacman::p_load("mvnrmtest")
# For visualizing hypothesis tests in multivariate linear models
pacman::p_load("heplots")
```

1 Generate the Synthetic Dataset

1.1 4 Algorithms, 10 independent datasets, and 5 independent runs

This simulates data collected from a Machine Learning model comparison experiment. The research question in this simulation is:

Is there a statistically significant difference in mean classification accuracy across the Machine Learning algorithms?

There are 4 algorithms, 10 independent datasets, and 5 independent runs per algorithm–dataset pair.

Execute the following R script to generate the synthetic dataset: `./data/synthetic-data-for-ml-model-compa`

“ $4 \times 10 \times 5 = 200$ observations” means:

- 4 algorithms
- 10 independent datasets
- 5 independent runs per algorithm–dataset pair

A “run” means:

- Different random seed
- Different initialization

- Possibly different train/test split

Therefore, the unit of analysis is the dataset, not the run. The 5 runs are replicates, not new experimental units.

1.2 NOT the same as Cross-Validation with Repeats

A run set up as a 10-Fold cross-validation with 5 repeats means:

For each algorithm on one dataset:

- The dataset is split into 10 folds
- The algorithm is trained and tested 10 times
- The entire 10-fold process is repeated 5 times

That gives 50 performance estimates per algorithm per dataset. However — and this is the critical part — those 50 values are **NOT independent observations**.

They are correlated because:

- The same data points appear in multiple folds
- Training sets overlap heavily
- Test sets are reused across repeats

This disqualifies it from being used in statistical tests of comparison that expect independence of observations.

1.3 Valid ways to Create Independent Datasets

1. Naturally distinct datasets

- Different real-world sources
- Different domains or tasks
- Examples: Different UCI datasets, different companies' sales datasets, different medical studies

2. Independent data-generating processes

- Simulated datasets with independent random draws
- Different parameterizations (noise level, class imbalance, feature correlations)

3. Independent experimental units

- Different time periods where stationarity holds

- Different geographical regions
- Different populations

2 Load the Dataset

```
ml_performance_data <- read_csv(
  "data/ml_results.csv",
  col_types = cols(
    Dataset = col_factor(
      levels = c("Dataset_1", "Dataset_2", "Dataset_3",
                 "Dataset_4", "Dataset_5", "Dataset_6",
                 "Dataset_7", "Dataset_8", "Dataset_9",
                 "Dataset_10")),
    Run      = col_factor(levels = c("1", "2", "3", "4", "5")),
    Algorithm = col_factor(levels = c("Logistic Regression",
                                      "Random Forest",
                                      "SVM", "Gradient Boosting")),
    Accuracy = col_double(),
    Precision = col_double(),
    Recall    = col_double()
  )
)

head(ml_performance_data)
```

```
#> # A tibble: 6 x 6
#>   Dataset Run Algorithm Accuracy Precision Recall
#>   <fct>   <fct> <fct>         <dbl>      <dbl>  <dbl>
#> 1 Dataset_1 1 Logistic Regression 0.703      0.747  0.702
#> 2 Dataset_2 1 Logistic Regression 0.713      0.739  0.690
#> 3 Dataset_3 1 Logistic Regression 0.767      0.761  0.754
#> 4 Dataset_4 1 Logistic Regression 0.722      0.733  0.722
#> 5 Dataset_5 1 Logistic Regression 0.724      0.716  0.737
#> 6 Dataset_6 1 Logistic Regression 0.771      0.762  0.738
```

3 Initial EDA

View the Dimensions

The number of observations and variables.

```
dim(ml_performance_data)
```

```
#> [1] 200 6
```

View the Data Types

```
sapply(ml_performance_data, class)
```

```
#> Dataset      Run Algorithm Accuracy Precision Recall  
#> "factor" "factor" "factor" "numeric" "numeric" "numeric"
```

```
str(ml_performance_data)
```

```
#> spc_tbl_ [200 x 6] (S3: spec_tbl_df/tbl_df/tbl/data.frame)  
#> $ Dataset : Factor w/ 10 levels "Dataset_1","Dataset_2",...: 1 2 3 4 5 6 7 8 9 10 ...  
#> $ Run      : Factor w/ 5 levels "1","2","3","4",...: 1 1 1 1 1 1 1 1 1 1 ...  
#> $ Algorithm: Factor w/ 4 levels "Logistic Regression",...: 1 1 1 1 1 1 1 1 1 1 ...  
#> $ Accuracy : num [1:200] 0.703 0.713 0.767 0.722 0.724 ...  
#> $ Precision: num [1:200] 0.747 0.739 0.761 0.733 0.716 ...  
#> $ Recall    : num [1:200] 0.702 0.69 0.754 0.722 0.737 ...  
#> - attr(*, "spec")=  
#> .. cols(  
#> .. Dataset = col_factor(levels = c("Dataset_1", "Dataset_2", "Dataset_3", "Dataset_4",  
#> .. "Dataset_6", "Dataset_7", "Dataset_8", "Dataset_9", "Dataset_10"  
#> .. ), ordered = FALSE, include_na = FALSE),  
#> .. Run = col_factor(levels = c("1", "2", "3", "4", "5"), ordered = FALSE, include_na = FALSE),  
#> .. Algorithm = col_factor(levels = c("Logistic Regression", "Random Forest", "SVM", "C",  
#> .. ), ordered = FALSE, include_na = FALSE),  
#> .. Accuracy = col_double(),  
#> .. Precision = col_double(),  
#> .. Recall = col_double()  
#> .. )  
#> - attr(*, "problems")=<externalptr>
```

Descriptive Statistics

Understanding your data can lead to:

- **Data cleaning:** To remove extreme outliers or impute missing data.

- **Data transformation:** To reduce skewness
- **Hypothesis formulation:** Formulate a hypothesis based on the patterns you identify
- **Choosing the appropriate statistical test:** You may notice properties of the data such as distributions or data types that suggest the use of parametric or non-parametric statistical tests and algorithms

Descriptive statistics can be used to understand your data. Typical descriptive statistics include:

1. **Measures of frequency:** count and percent
2. **Measures of central tendency:** mean, median, and mode
3. **Measures of distribution/dispersion/spread/scatter/variability:** minimum, quartiles, maximum, variance, standard deviation, coefficient of variation, range, interquartile range (IQR) [includes a box and whisker plot for visualization], kurtosis, skewness [includes a histogram for visualization].
4. **Measures of relationship:** covariance and correlation [includes a correlation plot and a scatter plot for visualization].

3.1 Measures of Frequency

```
ml_performance_data_freq <- ml_performance_data$Algorithm
cbind(frequency = table(ml_performance_data_freq),
      percentage = prop.table(table(ml_performance_data_freq)) * 100)
```

```
#>
#> frequency percentage
#> Logistic Regression      50      25
#> Random Forest           50      25
#> SVM                     50      25
#> Gradient Boosting       50      25
```

3.2 Measures of Central Tendency

The median and the mean of each numeric variable:

```
summary(ml_performance_data)
```

```

#>      Dataset  Run      Algorithm      Accuracy
#> Dataset_1:20 1:40 Logistic Regression:50 Min.    :0.6610
#> Dataset_2:20 2:40 Random Forest      :50 1st Qu.:0.7308
#> Dataset_3:20 3:40 SVM                  :50 Median  :0.7686
#> Dataset_4:20 4:40 Gradient Boosting  :50 Mean    :0.7647
#> Dataset_5:20 5:40                      3rd Qu.:0.7978
#> Dataset_6:20                      Max.    :0.9000
#> (Other)      :80
#>      Precision      Recall
#> Min.    :0.6465  Min.    :0.6585
#> 1st Qu.:0.7310  1st Qu.:0.7272
#> Median  :0.7633  Median  :0.7661
#> Mean    :0.7654  Mean    :0.7652
#> 3rd Qu.:0.7991  3rd Qu.:0.8029
#> Max.    :0.9000  Max.    :0.9000
#>

```

The first 5 observations (rows) in the dataset:

```
head(ml_performance_data, 5)
```

```

#> # A tibble: 5 x 6
#>   Dataset  Run  Algorithm      Accuracy Precision Recall
#>   <fct>    <fct> <fct>          <dbl>      <dbl>  <dbl>
#> 1 Dataset_1 1    Logistic Regression  0.703      0.747  0.702
#> 2 Dataset_2 1    Logistic Regression  0.713      0.739  0.690
#> 3 Dataset_3 1    Logistic Regression  0.767      0.761  0.754
#> 4 Dataset_4 1    Logistic Regression  0.722      0.733  0.722
#> 5 Dataset_5 1    Logistic Regression  0.724      0.716  0.737

```

The last 5 observations (rows) in the dataset:

```
tail(ml_performance_data, 5)
```

```

#> # A tibble: 5 x 6
#>   Dataset  Run  Algorithm      Accuracy Precision Recall
#>   <fct>    <fct> <fct>          <dbl>      <dbl>  <dbl>
#> 1 Dataset_6 5    Gradient Boosting  0.870      0.900  0.847
#> 2 Dataset_7 5    Gradient Boosting  0.828      0.813  0.825
#> 3 Dataset_8 5    Gradient Boosting  0.772      0.789  0.734
#> 4 Dataset_9 5    Gradient Boosting  0.792      0.766  0.815
#> 5 Dataset_10 5    Gradient Boosting  0.774      0.767  0.812

```


3.3 Measures of Distribution

Measuring the variability in the dataset is important because the amount of variability determines **how well you can generalize** results from the sample to a new observation in the population.

Low variability is ideal because it means that you can better predict information about the population based on the sample data. High variability means that the values are less consistent, thus making it harder to make predictions.

The syntax `dataset[rows, columns]` can be used to specify the exact rows and columns to be considered. `dataset[, columns]` implies all rows will be considered. For example, specifying `BostonHousing[, -4]` implies all the columns except column number 4. This can also be stated as `BostonHousing[, c(1,2,3,5,6,7,8,9,10,11,12,13,14)]`. This allows us to perform calculations on only columns that are numeric, thus leaving out the columns termed as “factors” (categorical) or those that have a string data type.

3.3.1 Variance

```
# `sapply()` is designed to apply a function to a variable in a dataset.  
# In this case, we use `sapply()` to apply the `var()` function used to  
# compute the variance.  
sapply(ml_performance_data[,c(4)], var)
```

```
#>    Accuracy  
#> 0.002014664
```

3.3.2 Standard Deviation

```
sapply(ml_performance_data[,c(4)], sd)
```

```
#>    Accuracy  
#> 0.04488501
```

3.3.3 Kurtosis (Pearson)

The Kurtosis informs us of how often outliers occur in the results. There are different formulas for calculating kurtosis. Specifying “type = 2” allows us to use the 2nd formula which is the same kurtosis formula used in other statistical software like SPSS and SAS. It is referred to as “Pearson’s definition of kurtosis”.

In “type = 2” (used in SPSS and SAS):

1. Kurtosis < 3 implies a low number of outliers → platykurtic
2. Kurtosis = 3 implies a medium number of outliers → mesokurtic
3. Kurtosis > 3 implies a high number of outliers → leptokurtic

High kurtosis (leptokurtic) affects models that are sensitive to outliers. Estimates of the variance are also inflated. Low kurtosis (platykurtic) implies a possible underestimation of real-world variability. The typical remedy includes trimming outliers or using robust statistical methods that are less affected by outliers.

```
pacman::p_load("e1071")  
sapply(ml_performance_data[,c(4)], kurtosis, type = 2)
```

```
#> Accuracy  
#> -0.4869903
```

3.3.4 Skewness

The skewness is used to identify the asymmetry of the distribution of results. Similar to kurtosis, there are several ways of computing the skewness.

Using “type = 2” (common in other statistical software like SPSS and SAS) can be interpreted as:

1. Skewness between -0.4 and 0.4 (inclusive) implies that there is no skew in the distribution of results; the distribution of results is symmetrical; it is a normal distribution; a Gaussian distribution.
2. Skewness above 0.4 implies a positive skew; a right-skewed distribution.
3. Skewness below -0.4 implies a negative skew; a left-skewed distribution.

Skewed data results in misleading averages and potentially biased model coefficients. The typical remedy to skewed data involves applying data transformations such as logarithmic, square-root, or Box-Cox, etc. to reduce skewness.

```
sapply(ml_performance_data[,c(4)], skewness, type = 2)
```

```
#> Accuracy  
#> 0.09007845
```

As a data analyst, you need to confirm if the distortion in kurtosis or skewness is a data problem or it is a real-world insight. For example, a real-world insight could be that few customers drive most of the value. This is as opposed to always looking at it as a distortion that needs to be corrected.

3.4 Basic Visualizations

3.4.1 Histogram

```
# `col_index` identifies which column(s) (by position) are being plotted;  
# seq_along() generates 1, 2, ..., ncol(data_frame) automatically,  
# so every column in the data frame is covered without manually listing  
# positions  
col_index <- seq_along(ml_performance_data)  
# the right-hand `col_index` (the full vector) is evaluated once when the  
# loop starts; col_index is then reassigned to one position per iteration.  
for (col_index in col_index) {  
  
  # skip silently, with no message/warning, if the position does not exist  
  # in the data (out of range) or if the column at that position is not  
  # numeric; `next` moves straight to the following value of col_index  
  if (col_index > ncol(ml_performance_data) ||  
      !is.numeric(ml_performance_data[[col_index]])) {  
    next  
  }  
  
  col_name <- names(ml_performance_data)[col_index]  
  
  # for a histogram, the continuous variable goes on x; geom_histogram()  
  # bins it and computes the count on y automatically, so y is left  
  # unmapped here (the reverse of the boxplot version, where x was a
```

```

# blank placeholder and y held the continuous variable)
p <- ggplot2::ggplot(ml_performance_data, aes(x = .data[[col_name]])) +
  geom_histogram(bins = 30, fill = "#4F6EA4", color = "#FFFFFF") +

# reduces the default 5% padding on the x and y axes
scale_x_continuous(expand = expansion(mult = c(0.00, 0.00))) +
scale_y_continuous(expand = expansion(mult = c(0.00, 0.00))) +
labs(
  title = paste0("Histogram of the Variable '", col_name, "'"),
  subtitle = paste0("EDA for distribution shape and outlier check ",
                    "before statistical tests and modelling"),
  x = col_name,
  y = "Count",
  caption = paste0("Source: ML Model Performance Data | n = ",
                    nrow(ml_performance_data), ", ",
                    "M = ",
                    sprintf("%.2f",
                              mean(ml_performance_data[[col_name]], na.rm = TRUE)),
                    ", ",
                    "sd = ",
                    sprintf("%.2f",
                              sd(ml_performance_data[[col_name]], na.rm = TRUE)))
) +
theme_minimal(base_size = 12) +

theme(
  plot.title      = element_text(face = "bold", color = "#1D1D1D"),
  axis.title      = element_text(color = "#1D1D1D"),
  axis.text       = element_text(color = "#898989"),
  axis.text.x     = element_text(angle = 30, hjust = 1),
  plot.caption    = element_text(color = "#898989"),
  # removes vertical gridlines
  panel.grid.major.x = element_blank(),
  panel.grid.minor.x = element_blank(),
  # horizontal gridlines are kept (at major breaks only) since they
  # help the reader judge bar heights against the y-axis scale
  panel.grid.minor.y = element_blank(),
  # draws only the x-axis and y-axis lines (the "L" frame)
  axis.line.x      = element_line(color = "#1D1D1D"),
  axis.line.y      = element_line(color = "#1D1D1D")
  # panel.border = element_rect(color = "#1D1D1D", fill = NA,
  #                               linewidth = 0.5)

```

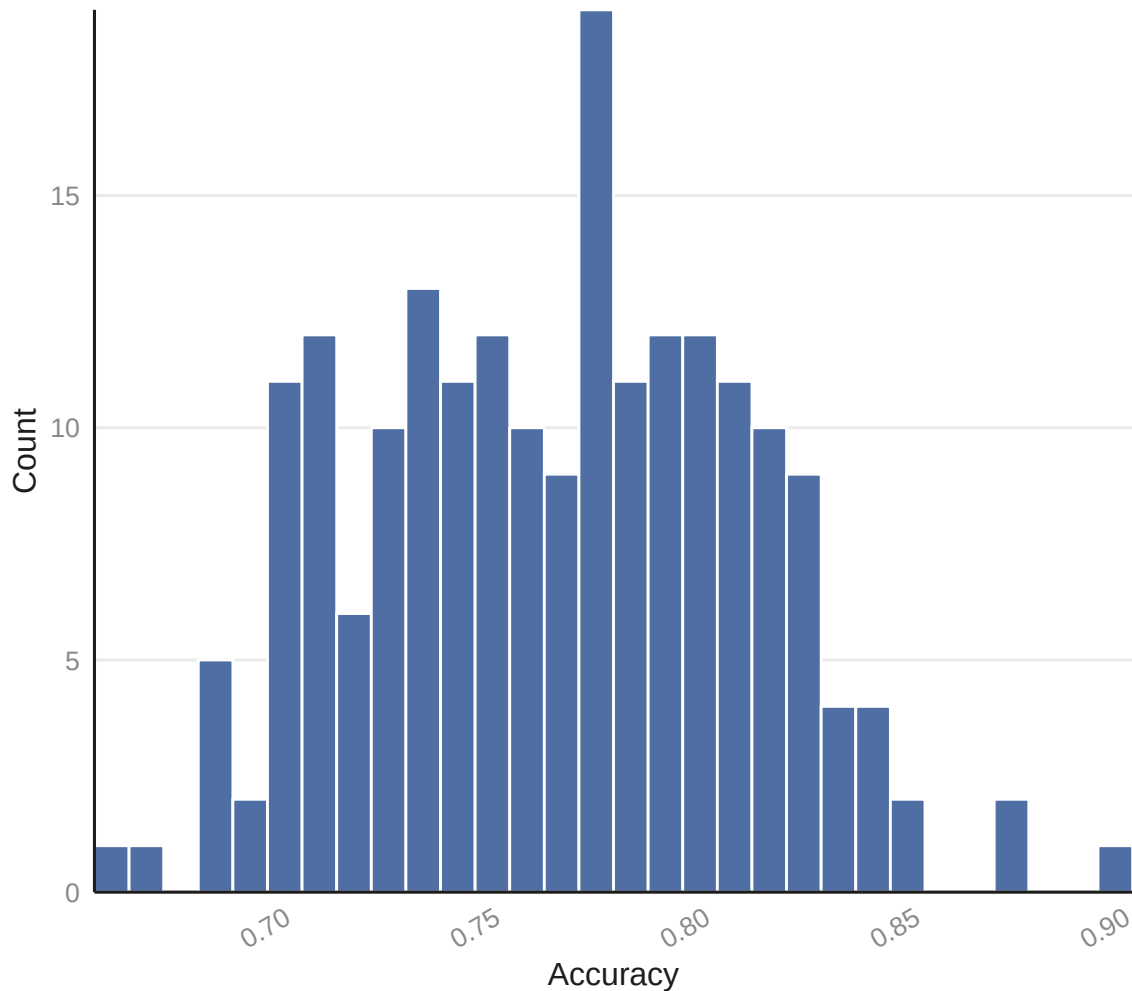
```

)
# ggplot objects do not auto-print inside a for-loop, therefore,
# each one must be printed explicitly otherwise it will never appear
print(p)
}

```

Histogram of the Variable 'Accuracy'

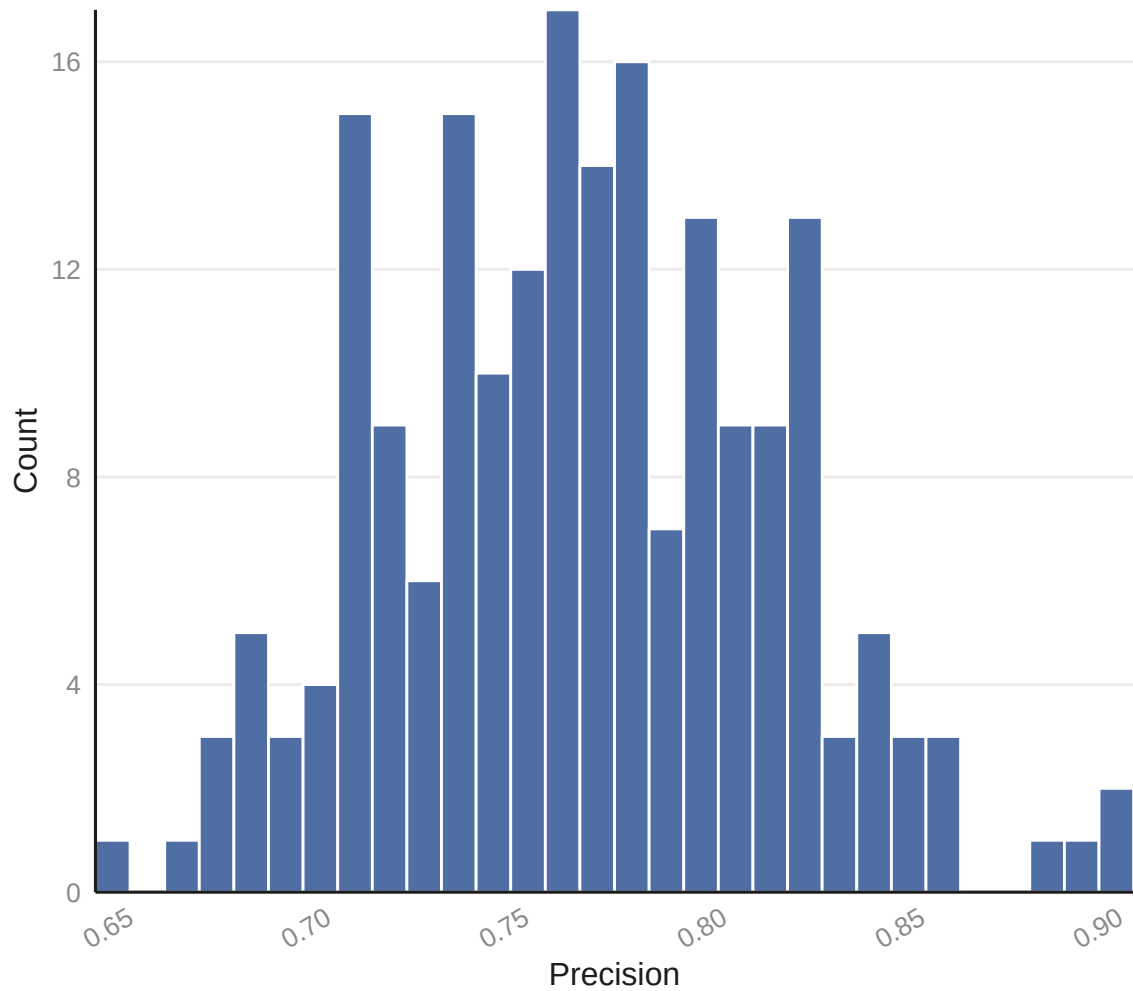
EDA for distribution shape and outlier check before statistical tests and mo



Source: ML Model Performance Data | n = 200, M = 0.76, sd = 0.04

Histogram of the Variable 'Precision'

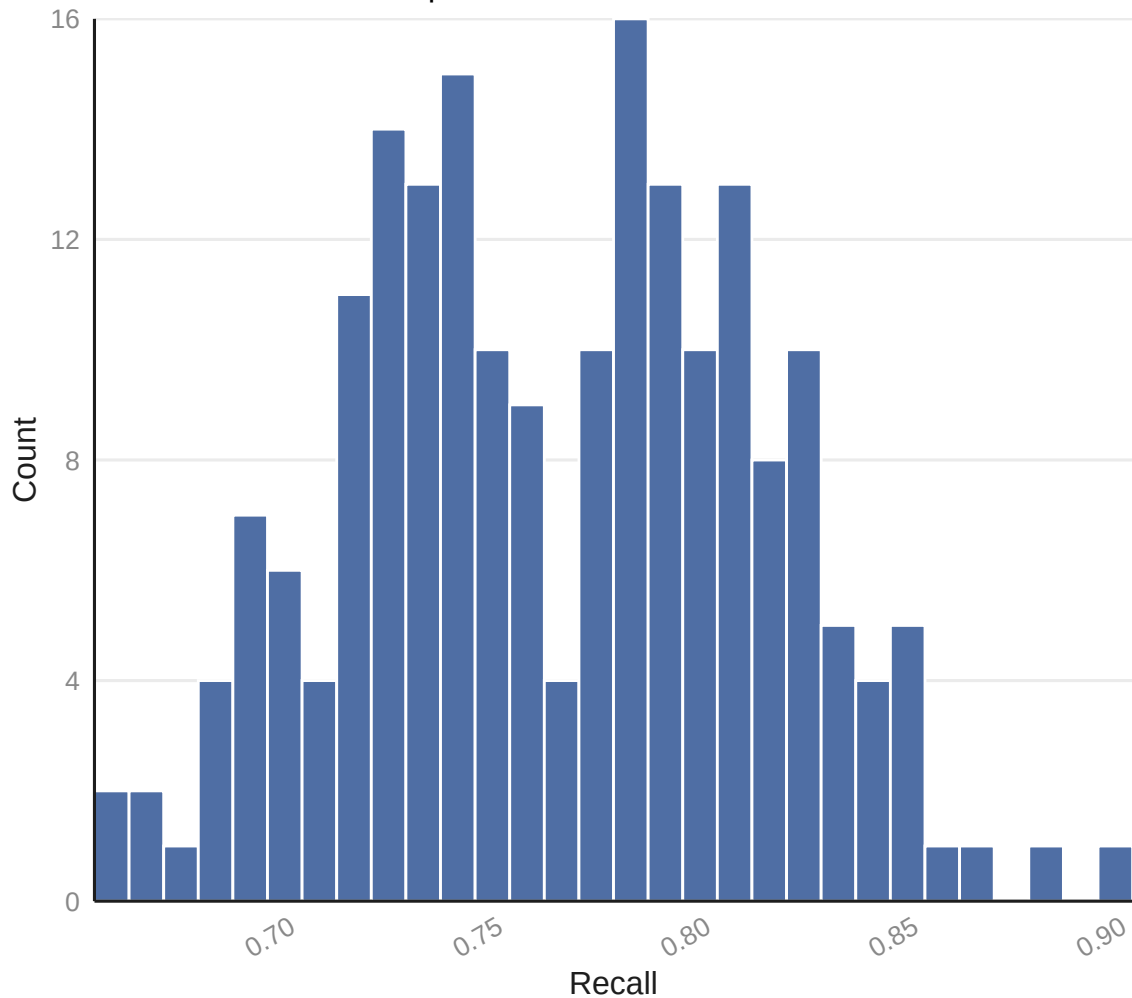
EDA for distribution shape and outlier check before statistical tests and mo



Source: ML Model Performance Data | n = 200, M = 0.77, sd = 0.05

Histogram of the Variable 'Recall'

EDA for distribution shape and outlier check before statistical tests and mo



Source: ML Model Performance Data | n = 200, M = 0.77, sd = 0.05

3.4.2 Box and Whisker Plot

```
# `col_index` identifies which column(s) (by position) are being plotted;  
# seq_along() generates 1, 2, ..., ncol(data_frame) automatically,  
# so every column in the data frame is covered without manually listing  
# positions  
col_index <- seq_along(ml_performance_data)
```

```

# the right-hand `col_index` (the full vector) is evaluated once when the
# loop starts; col_index is then reassigned to one position per iteration.
for (col_index in col_index) {

  # skip silently, with no message/warning, if the position does not exist
  # in the data (out of range) or if the column at that position is not
  # numeric; `next` moves straight to the following value of col_index
  if (col_index > ncol(ml_performance_data) ||
      !is.numeric(ml_performance_data[[col_index]])) {
    next
  }

  col_name <- names(ml_performance_data)[col_index]

  p <- ggplot2::ggplot(ml_performance_data, aes(x = "", y = .data[[col_name]])) +
    stat_boxplot(geom = "errorbar", width = 0.15, color = "#1D1D1D") +
    geom_boxplot(fill = "#4F6EA4", color = "#1D1D1D", width = 0.3) +

    # reduces the default 5% padding on the x and y axes
    # scale_x_continuous(expand = expansion(mult = c(0.00, 0.00))) +
    # scale_y_continuous(expand = expansion(mult = c(0.00, 0.00))) +
    labs(
      title = paste0("Box and Whisker Plot of the Variable '", col_name, "'"),
      subtitle = paste0("EDA for spread and outlier check before ",
                        "statistical tests and modelling"),
      x = NULL, # no x-axis label needed
      y = col_name,
      caption = paste0("Source: ML Model Performance Data | n = ",
                        nrow(ml_performance_data), ", ",
                        "M = ",
                        sprintf("%.2f",
                                mean(ml_performance_data[[col_name]], na.rm = TRUE)),
                        ", ",
                        "sd = ",
                        sprintf("%.2f",
                                sd(ml_performance_data[[col_name]], na.rm = TRUE)))
    ) +
    theme_minimal(base_size = 12) +

    theme(
      plot.title       = element_text(face = "bold", color = "#1D1D1D"),
      axis.title        = element_text(color = "#1D1D1D"),

```



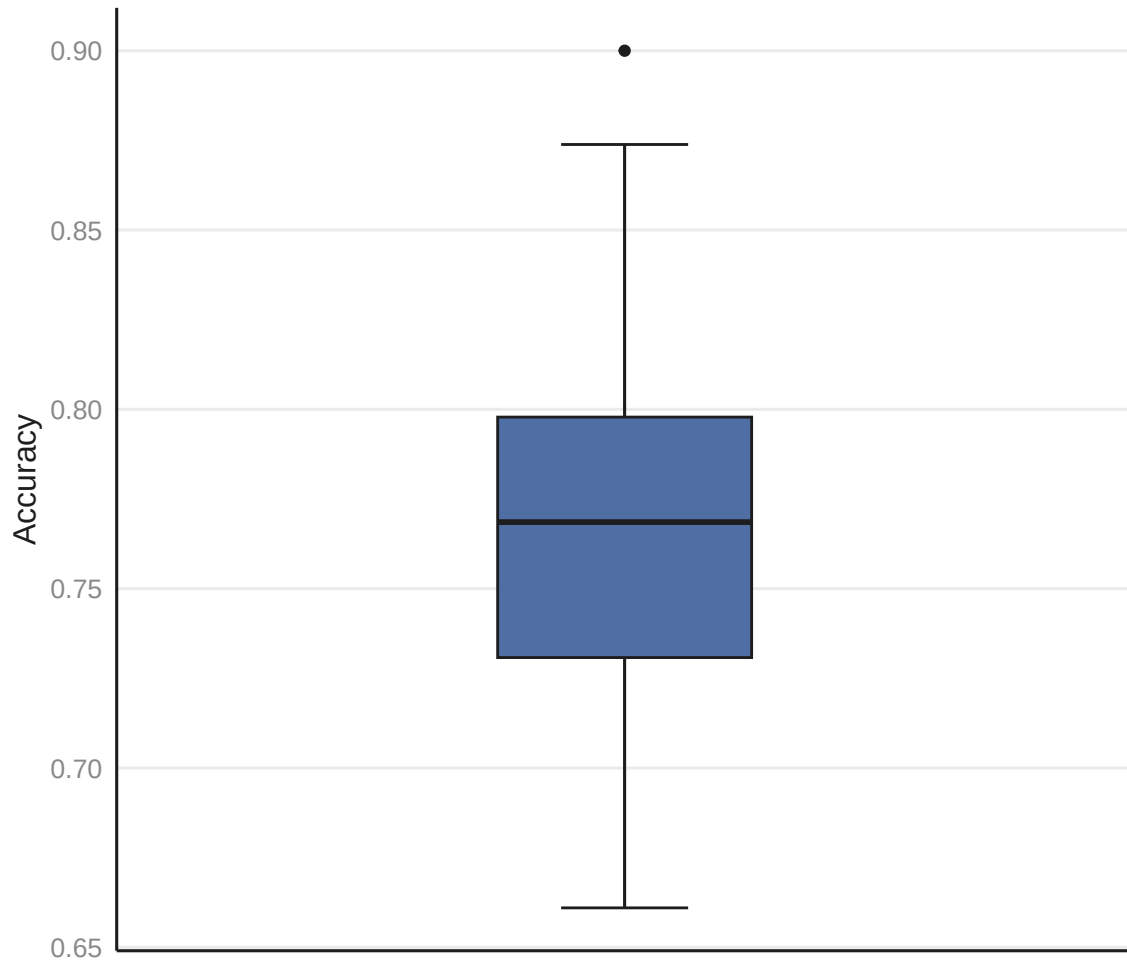
```

axis.text          = element_text(color = "#898989"),
# axis.text.x      = element_text(angle = 30, hjust = 1),
plot.caption      = element_text(color = "#898989"),
# removes vertical gridlines
panel.grid.major.x = element_blank(),
panel.grid.minor.x = element_blank(),
# horizontal gridlines are kept (at major breaks only) since they
# help the reader judge bar heights against the y-axis scale
panel.grid.minor.y = element_blank(),
# draws only the x-axis and y-axis lines (the "L" frame)
axis.line.x        = element_line(color = "#1D1D1D"),
axis.line.y        = element_line(color = "#1D1D1D")
# panel.border = element_rect(color = "#1D1D1D", fill = NA,
#                               linewidth = 0.5)
)
# ggplot objects do not auto-print inside a for-loop, therefore,
# each one must be printed explicitly otherwise it will never appear
print(p)
}

```

Box and Whisker Plot of the Variable 'Accuracy'

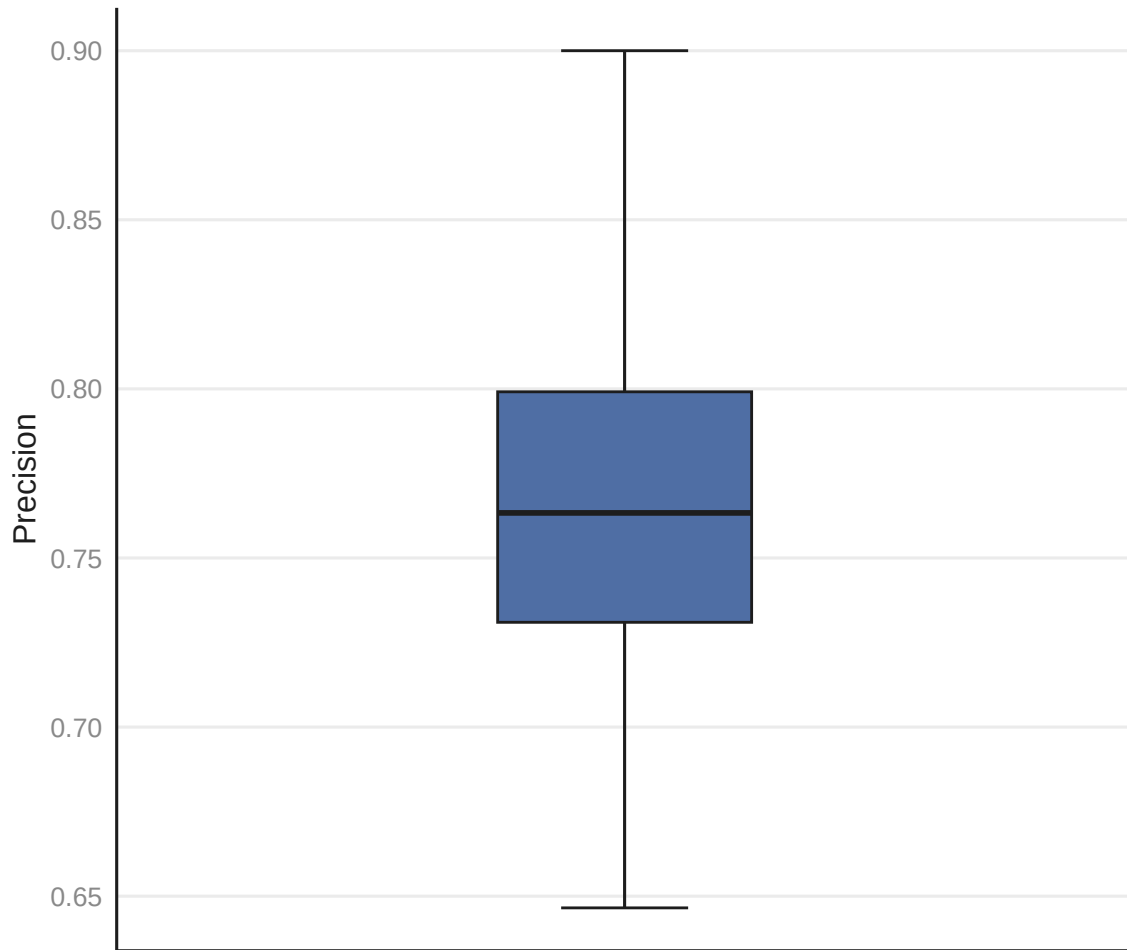
EDA for spread and outlier check before statistical tests and modelling



Source: ML Model Performance Data | $n = 200$, $M = 0.76$, $sd = 0.04$

Box and Whisker Plot of the Variable 'Precision'

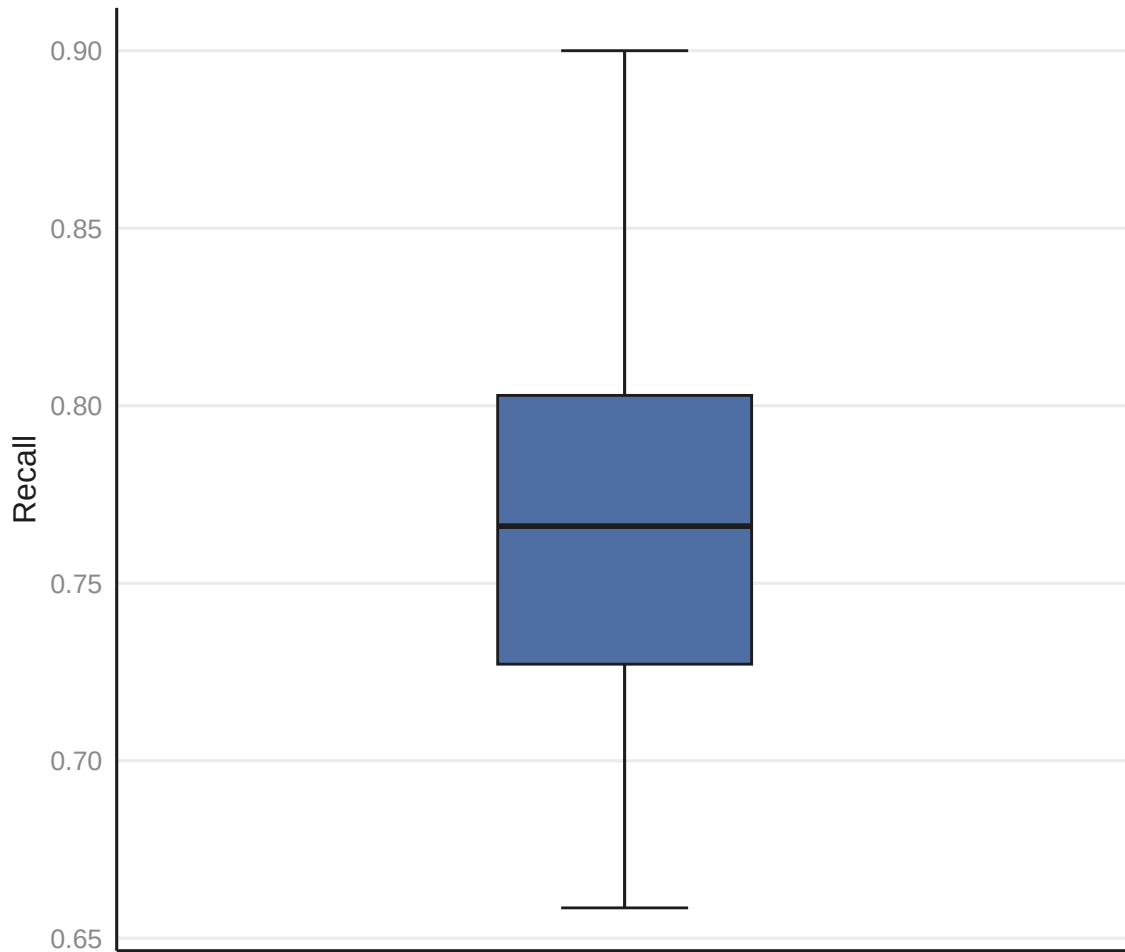
EDA for spread and outlier check before statistical tests and modelling



Source: ML Model Performance Data | $n = 200$, $M = 0.77$, $sd = 0.05$

Box and Whisker Plot of the Variable 'Recall'

EDA for spread and outlier check before statistical tests and modelling



Source: ML Model Performance Data | n = 200, M = 0.77, sd = 0.05

3.4.3 Missing Data Plot

```
# `vis_miss()` plots a matrix of observations (rows) by variables (columns),  
# shading each cell by whether the value is missing or present; this is  
# the ggplot2-native counterpart to Amelia's missmap()  
naniar::vis_miss(ml_performance_data) +  
  labs(  
    title = "Missing Data Overview",
```

```

    subtitle = "EDA for missingness pattern before data imputation or data removal",
    caption = paste0("Source: ML Model Performance Data | n = ",
                     nrow(ml_performance_data))
) +
# overrides vis_miss()'s default black/grey fill; vis_miss() has no
# built-in color argument, but since it returns a standard ggplot
# object, the fill scale can be replaced afterward like any other
scale_fill_manual(values = c("Present" = "#4F6EA4", "Missing" = "#E03C31")) +

theme_minimal(base_size = 12) +

theme(
  plot.title       = element_text(face = "bold", color = "#1D1D1D"),
  axis.title       = element_text(color = "#1D1D1D"),
  axis.text        = element_text(color = "#898989"),
  # axis.text.x    = element_text(angle = 30, hjust = 1),
  plot.caption     = element_text(color = "#898989"),
  # removes vertical gridlines
  panel.grid.major.x = element_blank(),
  panel.grid.minor.x = element_blank(),
  # horizontal gridlines are kept (at major breaks only) since they
  # help the reader judge bar heights against the y-axis scale
  panel.grid.minor.y = element_blank(),
  # draws only the x-axis and y-axis lines (the "L" frame)
  # axis.line.x     = element_line(color = "#1D1D1D"),
  # axis.line.y     = element_line(color = "#1D1D1D")
  # panel.border    = element_rect(color = "#1D1D1D", fill = NA,
  #                               # linewidth = 0.5)
)

```

Missing Data Overview

EDA for missingness pattern before data imputation or data removal



Source: ML Model Performance Data | n = 200

3.4.4 Scatter Plot

```
ggplot2::ggplot(ml_performance_data,  
                 aes(x = Algorithm, y = Accuracy)) +  
  # points use the blue-grey theme discussed in class,  
  # with partial transparency (alpha) so overlapping points are visible  
  # rather than solid black dots which obscure density.  
  geom_point(color = "#4F6EA4", alpha = 0.6, size = 2) +
```

```

# regression line in the near-black used for borders/axis text elsewhere,
# with a light blue-grey confidence ribbon instead of the default grey
geom_smooth(method = lm, color = "#1D1D1D", fill = "#A9C0DA") +
labs(
  title = "Relationship between Algorithm and Accuracy",
  subtitle = "EDA for linear association before modelling",
  x = "Algorithm",
  y = "Accuracy",
  caption = paste0("Source: ML Model Performance Data | n = ",
                    nrow(ml_performance_data))
) +

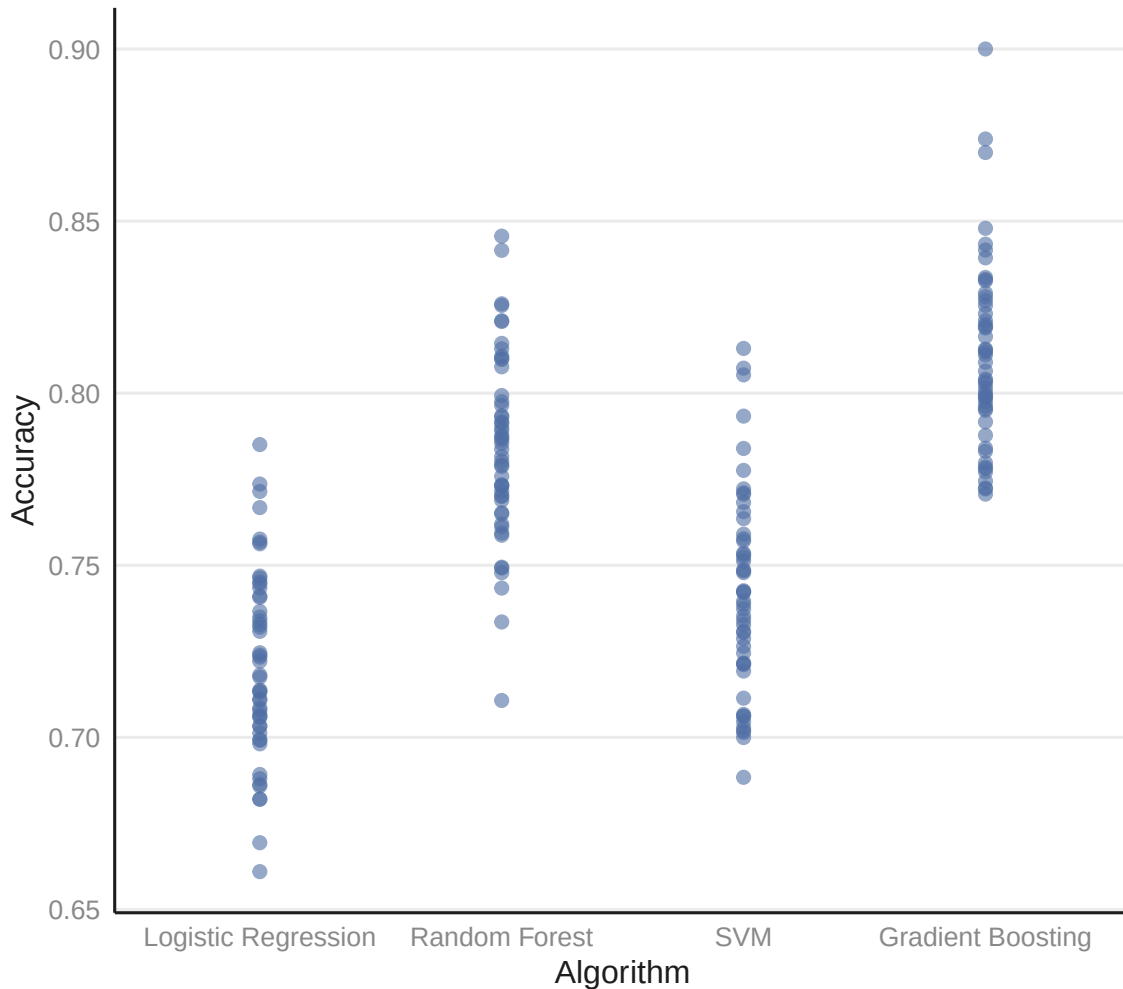
theme_minimal(base_size = 12) +

theme(
  plot.title          = element_text(face = "bold", color = "#1D1D1D"),
  axis.title          = element_text(color = "#1D1D1D"),
  axis.text           = element_text(color = "#898989"),
  # axis.text.x       = element_text(angle = 30, hjust = 1),
  plot.caption        = element_text(color = "#898989"),
  # removes vertical gridlines
  panel.grid.major.x  = element_blank(),
  panel.grid.minor.x  = element_blank(),
  # horizontal gridlines are kept (at major breaks only) since they
  # help the reader judge point position against the y-axis scale
  panel.grid.minor.y  = element_blank(),
  # draws only the x-axis and y-axis lines (the "L" frame)
  axis.line.x         = element_line(color = "#1D1D1D"),
  axis.line.y         = element_line(color = "#1D1D1D")
  # panel.border = element_rect(color = "#1D1D1D", fill = NA,
  # linewidth = 0.5)
)

```

Relationship between Algorithm and Accuracy

EDA for linear association before modelling



Source: ML Model Performance Data | n = 200

`geom_smooth(method = "lm")` fits a linear regression, which requires a continuous x-variable; `Algorithm` is categorical, therefore, this is not useful. Below is an alternative for a case where you have categorical data.

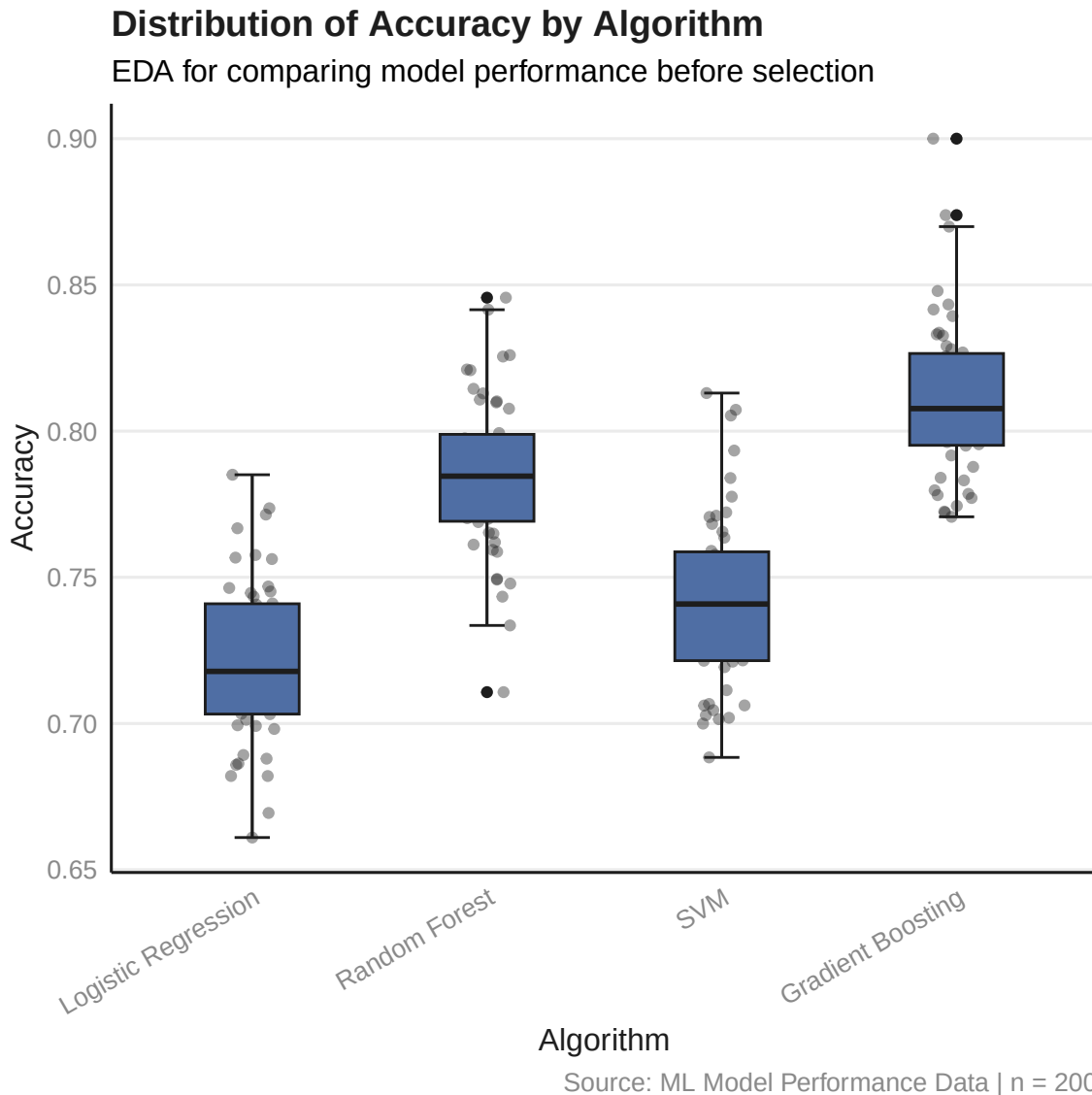
```
ggplot2::ggplot(ml_performance_data,
  aes(x = Algorithm, y = Accuracy)) +
  stat_boxplot(geom = "errorbar", width = 0.15, color = "#1D1D1D") +
  # layering the raw data points underneath the boxes with geom_jitter()
  # shows you both the summary statistics and the actual spread/sample size
  # per category, which a box alone hides, for instance you cannot tell
```



```

# from a box alone whether one category was tested on 5 runs and another
# on 50.
geom_jitter(width = 0.1, alpha = 0.4, color = "#1D1D1D", size = 1.5) +
# one box per category
geom_boxplot(fill = "#4F6EA4", color = "#1D1D1D", width = 0.4) +
labs(
  title = "Distribution of Accuracy by Algorithm",
  subtitle = "EDA for comparing model performance before selection",
  x = "Algorithm",
  y = "Accuracy",
  caption = paste0("Source: ML Model Performance Data | n = ",
                    nrow(ml_performance_data))
) +
theme_minimal(base_size = 12) +
theme(
  plot.title          = element_text(face = "bold", color = "#1D1D1D"),
  axis.title          = element_text(color = "#1D1D1D"),
  axis.text           = element_text(color = "#898989"),
  # uncomment if algorithm names are long enough to overlap when
  # printed horizontally (e.g. "Random Forest", "Gradient Boosting")
  axis.text.x         = element_text(angle = 30, hjust = 1),
  plot.caption        = element_text(color = "#898989"),
  # vertical gridlines removed: even with categorical groups on x,
  # there is no continuous scale for the lines to reference, so they
  # add no information
  panel.grid.major.x  = element_blank(),
  panel.grid.minor.x  = element_blank(),
  panel.grid.minor.y  = element_blank(),
  # draws only the x-axis and y-axis lines (the "L" frame)
  axis.line.x         = element_line(color = "#1D1D1D"),
  axis.line.y         = element_line(color = "#1D1D1D"),
  # panel.border = element_rect(color = "#1D1D1D", fill = NA,
  # linewidth = 0.5)
)

```



4 Statistical Test: ANOVA

4.1 One-Way ANOVA

Purpose: Compares the means of three or more groups defined by a single categorical independent variable, to test whether at least one group differs from the others.

Variables: One categorical independent variable (three or more levels); one continuous dependent variable.

Output: A single F-statistic and p-value; if significant, followed by a post-hoc test (e.g., Tukey's HSD) to identify which specific groups differ.

Example: Comparing Accuracy across four algorithms used to train ML models (accuracy depends on the chosen algorithm).

```
one_way_anova_result <- aov(Accuracy ~ Algorithm, data = ml_performance_data)
summary(one_way_anova_result)
```

```
#>               Df Sum Sq Mean Sq F value Pr(>F)
#> Algorithm         3  0.2469  0.08230   104.7 <2e-16 ***
#> Residuals       196  0.1540  0.00079
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Effect size addresses a gap that the F-test and p-value do not: those tell you whether the groups differ by more than chance, but not how much of the outcome's variability is actually attributable to the grouping factor. Eta squared (η^2) answers that second question. It is defined as the proportion of total variance in the dependent variable explained by the factor.

```
# effect size (eta squared): proportion of total variance explained by
# Algorithm; computed directly from the Sum Sq values already in the
# summary table, no extra package required
ss <- summary(one_way_anova_result)[[1]][["Sum Sq"]]
eta_squared_oneway <- ss[1] / sum(ss)
eta_squared_oneway
```

```
#> [1] 0.6158641
```

```
TukeyHSD(one_way_anova_result)
```

```
#>   Tukey multiple comparisons of means
#>     95% family-wise confidence level
#>
#> Fit: aov(formula = Accuracy ~ Algorithm, data = ml_performance_data)
#>
#> $Algorithm
#>
#>               diff             lwr             upr
#> Random Forest-Logistic Regression  0.06336014  0.048833192  0.07788709
#> SVM-Logistic Regression            0.02135088  0.006823931  0.03587783
#> Gradient Boosting-Logistic Regression 0.08998747  0.075460525  0.10451442
```

```

#> SVM-Random Forest -0.04200926 -0.056536211 -0.02748231
#> Gradient Boosting-Random Forest 0.02662733 0.012100383 0.04115428
#> Gradient Boosting-SVM 0.06863659 0.054109644 0.08316354
#> p adj
#> Random Forest-Logistic Regression 0.0000000
#> SVM-Logistic Regression 0.0010661
#> Gradient Boosting-Logistic Regression 0.0000000
#> SVM-Random Forest 0.0000000
#> Gradient Boosting-Random Forest 0.0000232
#> Gradient Boosting-SVM 0.0000000

```

4.2 Two-Way ANOVA

Purpose: Examines the effects of two categorical independent variables on a continuous outcome simultaneously, including whether the two variables interact, meaning the effect of one depends on the level of the other.

Variables: Two categorical independent variables; one continuous dependent variable.

Output: An F-statistic and p-value for each main effect, plus a separate F-statistic and p-value for the interaction effect; a significant interaction is interpreted before the main effects.

Example: Comparing Accuracy across Algorithm and Dataset, and testing whether an algorithm's relative performance changes depending on which dataset it was run on.

```

ml_performance_data$Algorithm <- as.factor(ml_performance_data$Algorithm)
ml_performance_data$Dataset <- as.factor(ml_performance_data$Dataset)

two_way_anova_result <- aov(Accuracy ~ Algorithm * Dataset, data = ml_performance_data)
shapiro.test(residuals(two_way_anova_result))

```

```

#>
#> Shapiro-Wilk normality test
#>
#> data: residuals(two_way_anova_result)
#> W = 0.99391, p-value = 0.587

```

```

leveneTest(Accuracy ~ Algorithm * Dataset, data = ml_performance_data)

```

```

#> Levene's Test for Homogeneity of Variance (center = median)
#>      Df F value Pr(>F)
#> group 39  0.9075 0.6285
#>      160

```

```
# `Algorithm * Dataset` tests both main effects and their interaction;
# a significant interaction would mean an algorithm's relative
# performance depends on which dataset it was run on, not just on
# the algorithm itself
summary(two_way_anova_result)
```

```
#>               Df Sum Sq Mean Sq F value Pr(>F)
#> Algorithm         3  0.24691  0.08230   99.175 <2e-16 ***
#> Dataset           9  0.00601  0.00067    0.805  0.612
#> Algorithm:Dataset 27  0.01521  0.00056    0.679  0.881
#> Residuals        160  0.13278  0.00083
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Effect size addresses a gap that the F-test and p-value do not: those tell you whether the groups differ by more than chance, but not how much of the outcome's variability is actually attributable to the grouping factor. Eta squared (η^2) answers that second question. It is defined as the proportion of total variance in the dependent variable explained by the factor.

```
ss <- summary(two_way_anova_result)[[1]][["Sum Sq"]]
row_names <- rownames(summary(two_way_anova_result)[[1]])

eta_squared_twoway <- ss / sum(ss)
names(eta_squared_twoway) <- trimws(row_names)
eta_squared_twoway
```

```
#>           Algorithm           Dataset Algorithm:Dataset           Residuals
#>           0.61586412           0.01500130           0.03794224           0.33119234
```

4.3 Multiple Analysis of Variance (MANOVA)

Purpose: Compares group means across two or more continuous dependent variables at the same time, accounting for the correlation between them, rather than testing each outcome separately.

Variables: One or more categorical independent variables; two or more continuous dependent variables, analyzed jointly.

Output: A multivariate test statistic (commonly Pillai's Trace); if significant, followed by univariate ANOVAs on each dependent variable individually to see which specific outcome(s) drove the result.

Example: Comparing Accuracy, Precision, and Recall together across Algorithm, rather than running three separate one-way ANOVAs.

```
# requires at least two continuous DVs; this will not run on the
# current Accuracy-only dataset and is included as the template for
# when additional metrics are added per run
manova_result <- manova(cbind(Accuracy, Precision, Recall) ~ Algorithm,
                        data = ml_performance_data)

# Pillai's trace is the recommended default test statistic, since it is
# the most robust to violations of multivariate normality and unequal
# covariance matrices among the four test statistics MANOVA offers
summary(manova_result, test = "Pillai")
```

```
#>           Df  Pillai approx F num Df den Df      Pr(>F)
#> Algorithm    3 0.66468   18.595      9   588 < 2.2e-16 ***
#> Residuals 196
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# univariate follow-up ANOVAs per outcome variable, the MANOVA
# equivalent of Tukey HSD follow-up after a significant one-way ANOVA
summary.aov(manova_result)
```

```
#> Response Accuracy :
#>           Df  Sum Sq Mean Sq F value      Pr(>F)
#> Algorithm    3 0.24691 0.082304  104.75 < 2.2e-16 ***
#> Residuals  196 0.15401 0.000786
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Response Precision :
#>           Df  Sum Sq Mean Sq F value      Pr(>F)
#> Algorithm    3 0.24990 0.083301   74.546 < 2.2e-16 ***
#> Residuals  196 0.21902 0.001117
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Response Recall :
#>           Df  Sum Sq Mean Sq F value      Pr(>F)
#> Algorithm    3 0.25127 0.083757   77.709 < 2.2e-16 ***
#> Residuals  196 0.21126 0.001078
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Effect size addresses a gap that the F-test and p-value do not: those tell you whether the groups differ by more than chance, but not how much of the outcome's variability is actually attributable to the grouping factor. Eta squared (²) answers that second question. It is defined as the proportion of total variance in the dependent variable explained by the factor.

```
manova_result <- manova(cbind(Accuracy, Precision, Recall) ~ Algorithm,
                        data = ml_performance_data)
```

```
aov_accuracy <- aov(Accuracy ~ Algorithm, data = ml_performance_data)
ss <- summary(aov_accuracy)[[1]][["Sum Sq"]]
eta_squared_accuracy <- ss[1] / sum(ss)
paste0("eta_squared_accuracy = ", eta_squared_accuracy)
```

```
#> [1] "eta_squared_accuracy = 0.615864122067398"
```

```
aov_precision <- aov(Precision ~ Algorithm, data = ml_performance_data)
ss <- summary(aov_precision)[[1]][["Sum Sq"]]
eta_squared_precision <- ss[1] / sum(ss)
paste0("eta_squared_precision = ", eta_squared_precision)
```

```
#> [1] "eta_squared_precision = 0.532930264223137"
```

```
aov_recall <- aov(Recall ~ Algorithm, data = ml_performance_data)
ss <- summary(aov_recall)[[1]][["Sum Sq"]]
eta_squared_recall <- ss[1] / sum(ss)
paste0("eta_squared_recall = ", eta_squared_recall)
```

```
#> [1] "eta_squared_recall = 0.543258295363704"
```

```
TukeyHSD(aov(Accuracy ~ Algorithm, data = ml_performance_data))
```

```
#> Tukey multiple comparisons of means
#> 95% family-wise confidence level
#>
#> Fit: aov(formula = Accuracy ~ Algorithm, data = ml_performance_data)
#>
#> $Algorithm
#>
#>           diff           lwr           upr
#> Random Forest-Logistic Regression 0.06336014 0.048833192 0.07788709
```

```

#> SVM-Logistic Regression          0.02135088  0.006823931  0.03587783
#> Gradient Boosting-Logistic Regression 0.08998747  0.075460525  0.10451442
#> SVM-Random Forest                -0.04200926 -0.056536211 -0.02748231
#> Gradient Boosting-Random Forest    0.02662733  0.012100383  0.04115428
#> Gradient Boosting-SVM             0.06863659  0.054109644  0.08316354
#>                                     p adj
#> Random Forest-Logistic Regression 0.00000000
#> SVM-Logistic Regression           0.0010661
#> Gradient Boosting-Logistic Regression 0.00000000
#> SVM-Random Forest                 0.00000000
#> Gradient Boosting-Random Forest    0.0000232
#> Gradient Boosting-SVM             0.00000000

```

```
TukeyHSD(aov(Precision ~ Algorithm, data = ml_performance_data))
```

```

#> Tukey multiple comparisons of means
#> 95% family-wise confidence level
#>
#> Fit: aov(formula = Precision ~ Algorithm, data = ml_performance_data)
#>
#> $Algorithm
#>                                     diff          lwr          upr
#> Random Forest-Logistic Regression 0.06851957  0.051195673  0.08584346
#> SVM-Logistic Regression           0.02156237  0.004238479  0.03888627
#> Gradient Boosting-Logistic Regression 0.08825802  0.070934123  0.10558191
#> SVM-Random Forest                 -0.04695719 -0.064281089 -0.02963330
#> Gradient Boosting-Random Forest    0.01973845  0.002414555  0.03706234
#> Gradient Boosting-SVM             0.06669564  0.049371749  0.08401954
#>                                     p adj
#> Random Forest-Logistic Regression 0.00000000
#> SVM-Logistic Regression           0.0079793
#> Gradient Boosting-Logistic Regression 0.00000000
#> SVM-Random Forest                 0.00000000
#> Gradient Boosting-Random Forest    0.0184291
#> Gradient Boosting-SVM             0.00000000

```

```
TukeyHSD(aov(Recall ~ Algorithm, data = ml_performance_data))
```

```

#> Tukey multiple comparisons of means
#> 95% family-wise confidence level
#>

```



```
#> Fit: aov(formula = Recall ~ Algorithm, data = ml_performance_data)
#>
#> $Algorithm
#>
#>               diff               lwr               upr
#> Random Forest-Logistic Regression  0.07021836  0.053204287  0.08723244
#> SVM-Logistic Regression            0.02320085  0.006186772  0.04021492
#> Gradient Boosting-Logistic Regression 0.08847608  0.071462005  0.10549016
#> SVM-Random Forest                 -0.04701752 -0.064031591 -0.03000344
#> Gradient Boosting-Random Forest      0.01825772  0.001243642  0.03527179
#> Gradient Boosting-SVM              0.06527523  0.048261157  0.08228931
#>
#>               p adj
#> Random Forest-Logistic Regression  0.0000000
#> SVM-Logistic Regression            0.0028565
#> Gradient Boosting-Logistic Regression 0.0000000
#> SVM-Random Forest                 0.0000000
#> Gradient Boosting-Random Forest      0.0301034
#> Gradient Boosting-SVM              0.0000000
```

5 Diagnostic EDA (Model Diagnostic)

5.1 Test of Linearity — Not Applicable

Linearity only exists between a predictor and outcome in a regression model. Group membership has no inherent order or scale, so there is no “relationship” between predictor and outcome whose shape (linear or otherwise) could be tested.

5.2 Test of Independence of Errors (Autocorrelation)

When observations are collected in a sequence (for example, repeated measurements over time, or runs executed one after another), the assumption of independence may be violated by autocorrelation: a correlation between residuals based on their order or proximity in the sequence. The Durbin-Watson test can be used to detect this. This test is not generally required for ANOVA designs without a natural ordering, since independence in those cases follows from random assignment and study design rather than from a statistical test.

Autocorrelation is consequential because, when positive, it leads to underestimated standard errors and inflated t-statistics, which can make findings appear more significant than they actually are. Negative autocorrelation has the opposite effect: standard errors become overestimated and tests become conservative.

A Durbin-Watson statistic close to 2 suggests no autocorrelation, while values approaching 0 indicate positive autocorrelation and values approaching 4 indicate negative autocorrelation.

For the Durbin-Watson test, as implemented by default in R's `dwtest()`:

- The null hypothesis, H_0 , is that there is no autocorrelation ($\rho = 0$).
- The alternative hypothesis, H_a , is that there is positive autocorrelation ($\rho > 0$).

By default, `dwtest()` tests only for positive autocorrelation, since this is the form most commonly encountered in sequential data. If a test against negative autocorrelation, or against autocorrelation in either direction, is needed instead, the `alternative` argument can be set to "less" or "two.sided", respectively.

If the p-value of the Durbin-Watson statistic is greater than 0.05, there is no evidence to reject the null hypothesis of no autocorrelation.

Example: Algorithm benchmarks run sequentially on the same machine, where thermal throttling, caching, or background load could make consecutive runs correlated.

```
lmtest::dwtest(one_way_anova_result)
```

```
#>
#> Durbin-Watson test
#>
#> data:  one_way_anova_result
#> DW = 2.1638, p-value = 0.8292
#> alternative hypothesis: true autocorrelation is greater than 0
```

```
lmtest::dwtest(two_way_anova_result)
```

```
#>
#> Durbin-Watson test
#>
#> data:  two_way_anova_result
#> DW = 2.1142, p-value = 0.7862
#> alternative hypothesis: true autocorrelation is greater than 0
```

```
lmtest::dwtest(manova_result)
```

```
#>
#> Durbin-Watson test
#>
#> data:  manova_result
#> DW = 2.1559, p-value = 0.8146
#> alternative hypothesis: true autocorrelation is greater than 0
```

Interpretation: The Durbin-Watson statistic for one-way ANOVA ($DW = 2.1638$), two-way ANOVA ($DW=2.1142$), and multiple analysis of variance (MANOVA) ($DW=2.1559$) are all close to 2, indicating little evidence of autocorrelation among the residuals. The one-way ANOVA p-value (0.8292), two-way ANOVA p-value (0.7862), and MANOVA p-value (0.8146) are far greater than 0.05, so there is no evidence to reject the null hypothesis. The assumption of independent errors is therefore reasonably supported.

5.3 Test of Normality of the Distribution of the Errors

The Shapiro–Wilk test assesses whether a sample of data could reasonably have come from a normally distributed population. It enables you to answer the question, “Is this data “normal enough” for methods that assume normality?”

It tests the following hypothesis:

H : The population from which the sample is drawn is normally distributed.

H : The population from which the sample is drawn is not normally distributed.

The test compares your ordered data values to the values you would expect if the data were perfectly normal. The more they disagree, the lower the Shapiro–Wilk statistic (W), and the smaller the p-value.

It uses the same logic as a Q–Q plot, but expressed mathematically.

Interpretation

If p-value > 0.05

You fail to reject the null hypothesis. There is no evidence that the data deviates from a normal distribution. However, this does not mean the data is truly normal. It simply means it is “normal enough” for the chosen statistical test of hypothesis.

Sample Size Matters

- Small samples ($n < 30$): Shapiro–Wilk has low power. It often fails to detect non-normality even when it is present.
- Large samples ($n > 50$): Shapiro–Wilk gains enough power to detect very small, practically negligible deviations from normality. A significant result at large n does not necessarily mean the deviation matters for your analysis; ANOVA is fairly robust to mild non-normality when samples are large. Pairing the test with a Q–Q plot helps judge whether a detected deviation is practically meaningful.

```
shapiro.test(residuals(one_way_anova_result))
```

```
#>
#> Shapiro-Wilk normality test
#>
#> data: residuals(one_way_anova_result)
#> W = 0.98827, p-value = 0.09886
```

```
shapiro.test(residuals(two_way_anova_result))
```

```
#>
#> Shapiro-Wilk normality test
#>
#> data: residuals(two_way_anova_result)
#> W = 0.99391, p-value = 0.587
```

```
result <- MVN::mvn(data = residuals(manova_result), mvn_test = "mardia")
print(result$multivariate_normality)
```

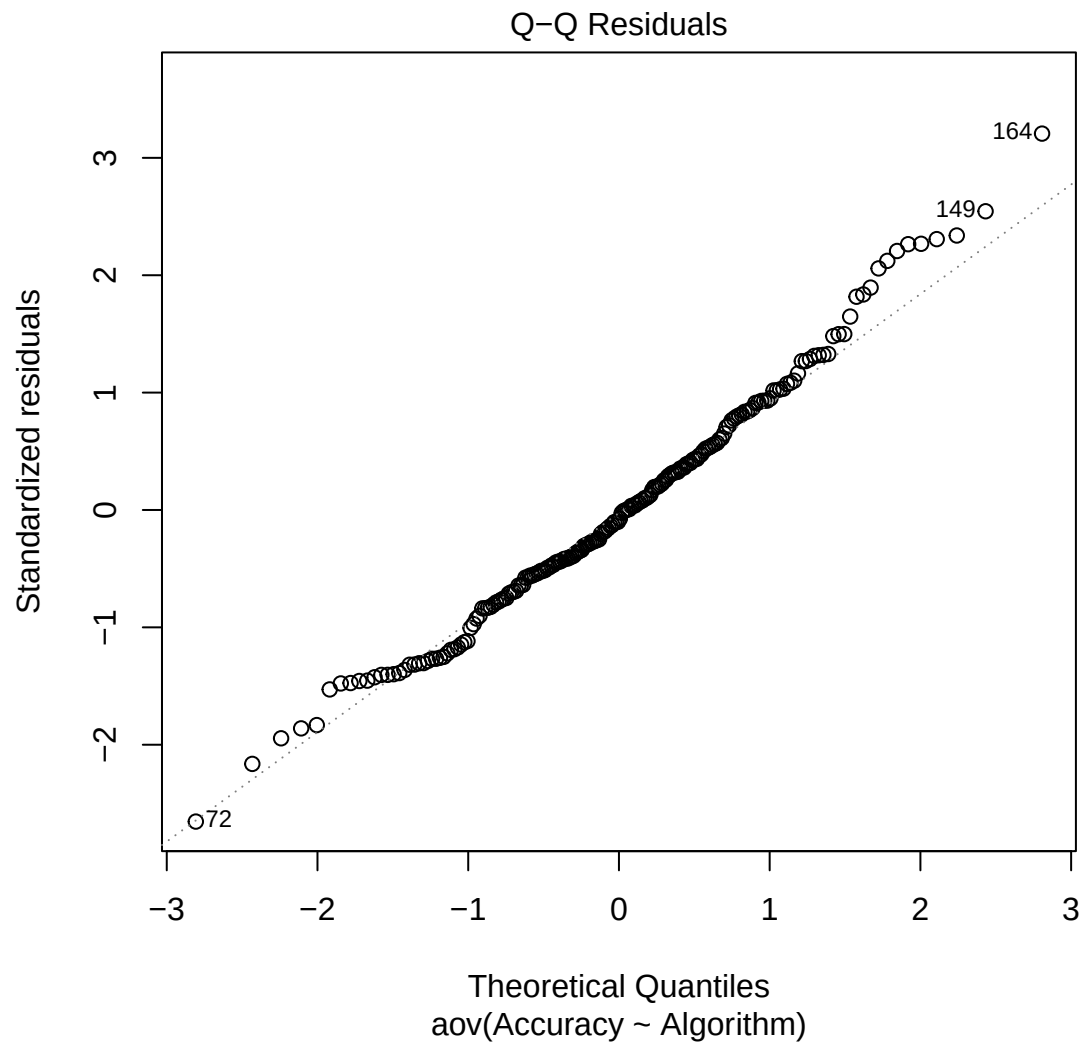
```
#>
#>      Test Statistic p.value      Method      MVN
#> 1 Mardia Skewness    10.708    0.381 asymptotic Normal
#> 2 Mardia Kurtosis     0.277    0.782 asymptotic Normal
```

The test of normality of the distribution of the errors assesses whether the errors (residuals) are approximately normally distributed, i.e., most errors are close to zero and large errors are rare. A Q-Q plot can be used to conduct the test of normality.

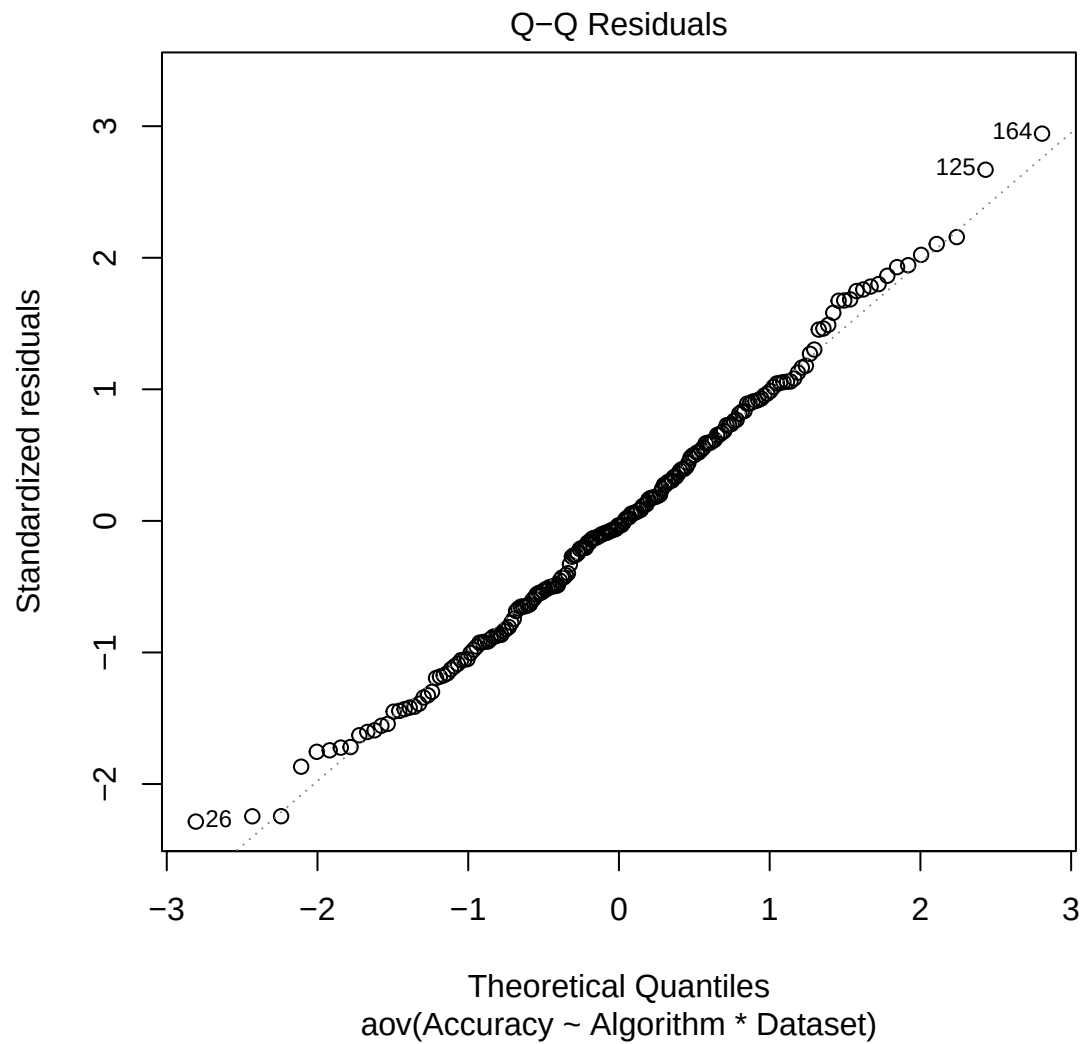
A Q-Q plot is a scatterplot of the quantiles of the errors against the quantiles of a normal distribution. Quantiles are statistical values that divide a dataset or probability distribution into equal-sized intervals. They help in understanding how data is distributed by marking specific points that separate the data into groups of equal size. Examples of quantiles include: quartiles (4 equal parts), percentiles (100 equal parts), deciles (10 equal parts), etc.

If the points in the Q-Q plot fall along a straight line, then the normality assumption is satisfied. If the points in the Q-Q plot do not fall along a straight line, then the normality assumption is not satisfied.

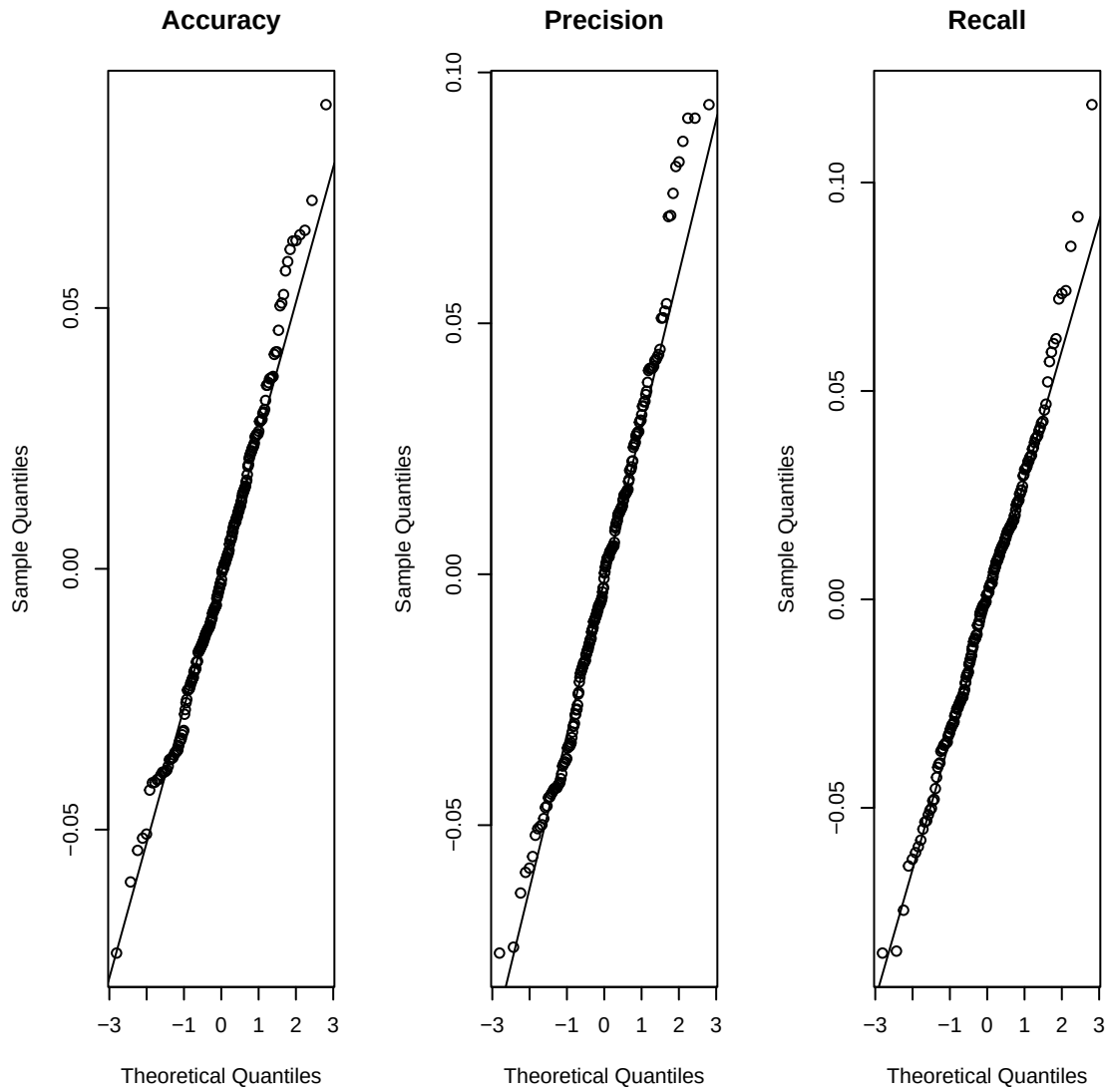
```
plot(one_way_anova_result, which = 2)
```



```
plot(two_way_anova_result, which = 2)
```



```
resid_matrix <- residuals(manova_result)
par(mfrow = c(1, ncol(resid_matrix)))
for (i in 1:ncol(resid_matrix)) {
  qqnorm(resid_matrix[, i], main = colnames(resid_matrix)[i])
  qqline(resid_matrix[, i])
}
```



```
par(mfrow = c(1, 1))
```

Interpretation: The Shapiro-Wilk statistic for one-way ANOVA ($W = 0.98827$, $p = 0.09886$) and two-way ANOVA ($W = 0.99391$, $p = 0.587$) both yield p-values greater than 0.05, so there is no evidence to reject the null hypothesis of normality for either model; the assumption of normally distributed residuals is reasonably supported in both cases.

For MANOVA, residuals were instead evaluated using Mardia's multivariate normality test, since MANOVA's normality assumption concerns the joint distribution of residuals across all response variables rather than each variable individually. Mardia's skewness statistic (10.708,

$p = 0.381$) and kurtosis statistic (0.277, $p = 0.782$) both yield p-values well above 0.05, so there is no evidence to reject the null hypothesis of multivariate normality. The assumption of normally distributed residuals is therefore reasonably supported for the MANOVA model as well.

`shapiro.test()` evaluates only a single variable's residuals at a time, while MANOVA's normality assumption is inherently multivariate. The Mardia test, which properly accounts for the joint distribution, is the appropriate diagnostic here and indicates that the assumption holds.

5.4 Test of Homoscedasticity (Homogeneity of Variance)

Homoscedasticity (homogeneity of variance) means that the variability of the dependent variable is approximately the same across all groups of the independent variable.

Levene's Test is used in cases where the dependent variable is numeric and the independent variable is categorical.

It tests the following hypothesis:

H : The variance of the dependent variable is equal across all levels of the independent variable.

H : At least one level of the independent variable has a different variance.

If the p-value is greater than 0.05, we fail to reject the null hypothesis and conclude that there is insufficient evidence to suggest that the group variances differ.

The purpose of checking variances is not only to satisfy a statistical assumption. Variance can provide important practical insights about the consistency or stability of different groups.

A group with a high variance may indicate greater uncertainty, inconsistency, instability, or risk compared with other groups. The interpretation depends on the application domain.

```
car::leveneTest(Accuracy ~ Algorithm, data = ml_performance_data)
```

```
#> Levene's Test for Homogeneity of Variance (center = median)
#>      Df F value Pr(>F)
#> group  3  0.2559  0.857
#>      196
```

Meaning of the degrees of freedom from the one-way anova:

- 3 = between-group degrees of freedom, suggesting there are 4 algorithms ($k - 1 = 3$)
- 196 = within-group (error) degrees of freedom

This implies a total sample size of approximately:

- $N = 196 + 4 = 200$

assuming no missing values.

```
car::leveneTest(Accuracy ~ Algorithm * Dataset, data = ml_performance_data)
```

```
#> Levene's Test for Homogeneity of Variance (center = median)
#>      Df F value Pr(>F)
#> group 39  0.9075 0.6285
#>      160
```

MANOVA has multiple response variables, and the assumption it actually requires is homogeneity of the *variance-covariance matrices* across groups, not homogeneity of variance for each outcome individually.

```
```{r test_of_homoscedasticity_manova}
heplots::boxM(cbind(Accuracy, Precision, Recall) ~ Algorithm,
 data = ml_performance_data)
```
```

```
#>
#> Box's M-test for Homogeneity of Covariance Matrices
#>
#> data:  ml_performance_data
#> Chi-Sq (approx.) = 12.3601, df = 18, p-value = 0.8281
```

Interpretation:

For the one-way ANOVA, Levene's test indicated that the variances of accuracy scores were not significantly different across the algorithms, $F(3, 196) = 0.256$, $p = .857$. The assumption of homogeneity of variance was therefore satisfied.

For the two-way ANOVA, Levene's test was conducted across all combinations of Algorithm and Dataset to assess homogeneity of variance within each cell of the design. The result was non-significant, $F(39, 160) = 0.908$, $p = .629$, indicating that the assumption of homogeneity of variance was satisfied across the crossed design.

For the MANOVA, homogeneity of variance-covariance matrices across Algorithm groups was assessed using Box's M test, since MANOVA's assumption concerns the joint covariance structure across all dependent variables (Accuracy, Precision, Recall) rather than the variance of

each outcome individually. The result was non-significant, $\chi^2(18) = 12.360$, $p = .828$, indicating no evidence against the assumption of homogeneous covariance matrices across groups. The assumption was therefore reasonably supported.

Box's M test is known to be highly sensitive to departures from multivariate normality and tends to reject the null more readily as sample size increases; a non-significant result here is reassuring, but a significant result would not necessarily indicate a fatal violation, particularly with reasonably balanced group sizes, given MANOVA's general robustness to this assumption under such conditions.

5.5 Quantitative Validation of Assumptions — Not Applicable

Although `aov()` objects do generate residuals and can technically be passed to `gvlma()`, the function's link function (linearity) component is not meaningful here, since the independent variable is a categorical factor with no inherent order or numeric scale. Quantitative validation is instead carried out using assumption-specific tests: `shapiro.test()` for normality, `leveneTest()` for homogeneity of variance, and, where observations are sequentially ordered, `dwtest()` for independence.

6 Interpretation of the Results

The presentation of the results and its subsequent interpretation is based on the following notes.

t-Statistic—t(d.f.): In a one-sample t-test, the t-statistic measures how far the sample mean is from the hypothesised population mean.

- A t-value close to 0 indicates that the sample mean is very close to the hypothesised value.
- A larger absolute t-value indicates that the observed mean is many standard errors away from the hypothesised mean, providing stronger evidence against the null hypothesis.

The t-statistic has an associated p-value, which quantifies how likely it is to observe a t-value this extreme (or more extreme) if the null hypothesis were true.

Degrees of Freedom—(d.f.): Degrees of freedom refers to the number of values in a calculation that are free to vary. It is essentially a measure of how much independent information is available for estimating a statistical parameter.

For example: Imagine you need to calculate the average height of 5 people, and you know the sum of all their heights is 340 inches. If you know the heights of 4 of these people (65, 70, 68, and 72 inches), you can automatically determine the height of the fifth person without measuring them: $340 - (65 + 70 + 68 + 72) = 65$ inches. In this example, even though there

are 5 people, you only have 4 degrees of freedom because once you know 4 heights and the total, the 5th height is no longer “free to vary” – it is determined by the other values.

Confidence Interval

A 95% confidence interval (CI) for a parameter provides a range that, under repeated sampling, would contain the true (but unknown) population parameter 95% of the time. Analogy: Imagine shooting arrows at a target. If you drew a circle around where 95% of your arrows landed, that circle is like a confidence interval—it captures the region in which your “shots” (i.e., estimates from different samples) tend to fall.

Uncertainty quantification: A CI communicates your estimate’s precision—narrower intervals imply more precise estimates (often due to larger samples or less variability), whereas wider intervals indicate greater uncertainty about the true value.

Academic Reporting (based on the APA 7th Edition Style)

Below are some key considerations to note when reporting statistical analysis using the APA style:

1. The type of statistical test must be stated.
2. Although not mandatory, the dependent variable is usually stated first followed by the independent variable when describing relationships, e.g., “...to examine whether advertising expenditures on YouTube, TikTok, and Facebook collectively predict Sales” such that Sales is the dependent variable that depends on advertising expenditures on YouTube, TikTok, and Facebook.
3. Test statistic and parameters: Report the appropriate test statistic (*t*-Statistic, *F*-Statistic, χ^2 , etc.) with the degrees of freedom in parentheses. The italicized statistical symbol is immediately followed by the degrees of freedom in parentheses without a space, e.g., *t*(498) and not *t* (498).
4. Exact p-values: Report exact p-values, when possible (e.g., $p = .032$), unless they are less than .001, then report as $p < .001$.
5. Effect sizes: Include appropriate effect size measures (e.g., R^2) to indicate practical significance.
6. Standard errors: Report standard errors of estimates when relevant. The standard error tells you how much your estimate might vary if you were to repeat your study with different random samples from the same population. A smaller standard error indicates a more precise estimate.
7. Confidence Intervals (CI): The confidence level should be clearly stated whenever you report point estimates (e.g., means, regression coefficients, correlations, etc.). The 95% confidence interval is the most common, and if another level is used (e.g., 90% CI, 99% CI), it should be explicitly mentioned. Confidence intervals are typically enclosed in

square brackets [], with the lower and upper limits separated by a comma. For example: 95% CI [-.03, .04]. They are usually reported directly after the statistic they describe, often within the same sentence or in parentheses.

8. Two decimal places: Report to two decimal places, except p-values which may need three or more decimal places. Round to the nearest value; do not truncate.
9. Descriptive statistics: Report relevant means, standard deviations, and sample sizes, e.g., The sample size included 500 observations ($M = 25.43$, $SD = 4.62$).
10. Italicize statistical symbols: Use italics for statistical symbols (t , F , p , etc.) but not for Greek letters (μ , σ , α), subscripts, or parenthetical information, e.g., $R^2 = .45$, $F(2, 97) = 15.62$, $p < .001$, The participants ($N = 120$) had an average score ($M = 25.43$, $SD = 4.62$) on the cognitive test.

Further reading: <https://apastyle.apa.org/jars>

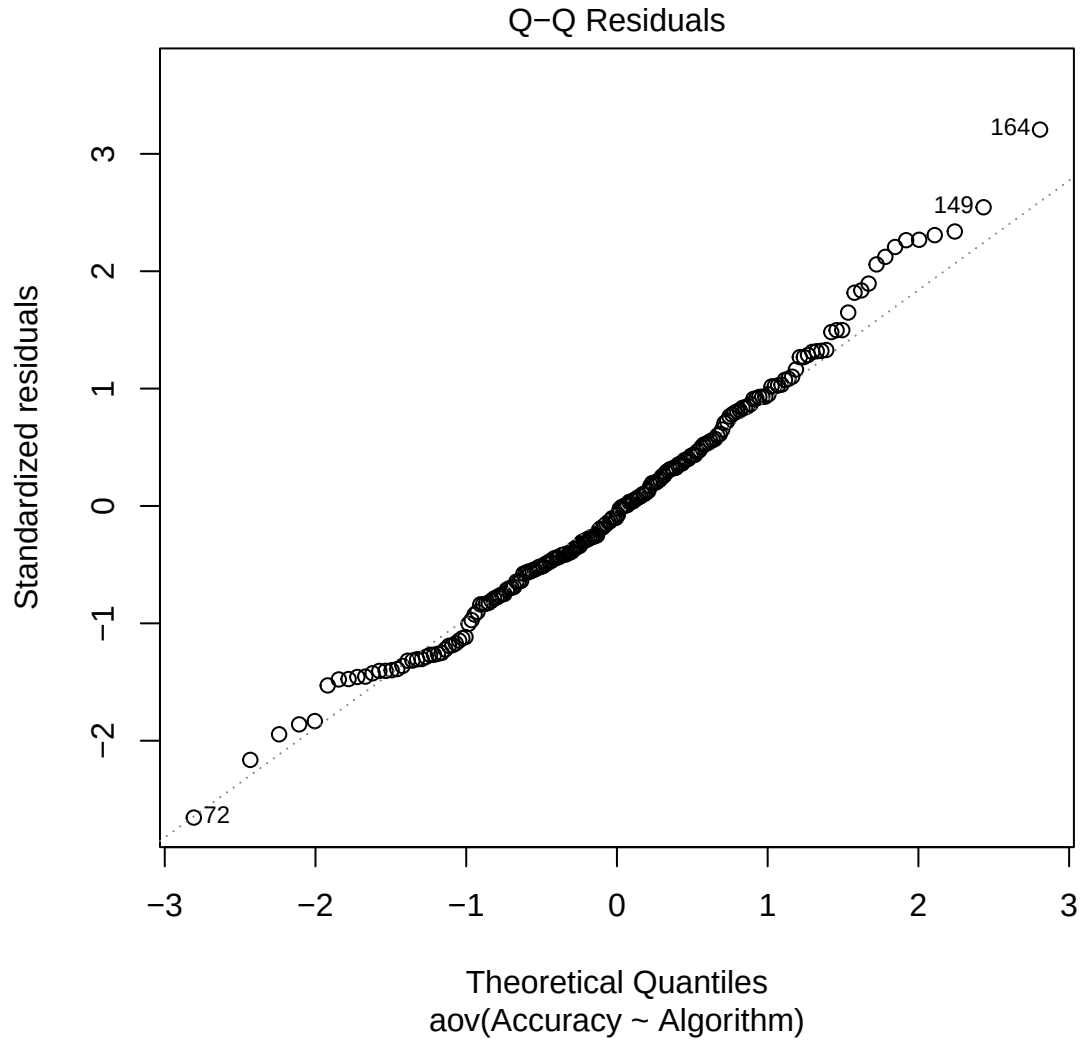
6.1 One-Way Analysis of Variance

6.1.1 Technical Academic Report (Based on APA Guidelines of Reporting Results)

A one-way analysis of variance was conducted to compare the effect of the chosen algorithm (Logistic Regression, Random Forest, Support Vector Machine, Gradient Boosting) on classification accuracy. The effect of the chosen algorithm was statistically significant, $F(3, 196) = 104.70$, $p < .001$, $\eta^2 = .62$, indicating that algorithm choice accounted for approximately 62% of the variance in accuracy — a large effect by conventional benchmarks.

An omnibus test is a statistical test that evaluates whether there is a significant effect or difference somewhere among three or more groups or variables, without identifying which specific groups or pairs are driving that effect. The term “omnibus” (Latin for “for all”) reflects that the test addresses a single overall question across the entire set of groups simultaneously, rather than each pairwise comparison individually.

Prior to interpreting the omnibus test, assumptions were checked. Levene’s test indicated no violation of homogeneity of variance, $F(3, 196) = 0.26$, $p = .857$. The Shapiro-Wilk test indicated that residuals did not significantly depart from normality, $W = .99$, $p = .099$, though this value approaches the conventional threshold and warrants visual confirmation given the sample size. See the figure below for the Q-Q plot that provides a visual confirmation.



A Durbin-Watson test found no evidence of positive autocorrelation in the residuals, $DW = 2.16$, $p = .829$; this result is meaningful only insofar as the row order of the dataset reflects a genuine sequence.

Post hoc comparisons using Tukey's HSD test were conducted to examine all pairwise differences. Every comparison was statistically significant, producing a complete and unambiguous ranking: Gradient Boosting > Random Forest > Support Vector Machine > Logistic Regression.

| Comparison | Mean Diff. | 95% CI | p (adj.) | Cohen's d^1 |
|---|------------|----------------|------------|---------------|
| Gradient Boosting vs. Logistic Regression | 0.090 | [0.075, 0.105] | < .001 | 3.20 |
| Gradient Boosting vs. SVM | 0.069 | [0.054, 0.083] | < .001 | 2.44 |
| Gradient Boosting vs. Random Forest | 0.027 | [0.012, 0.041] | < .001 | 0.95 |
| Random Forest vs. Logistic Regression | 0.063 | [0.049, 0.078] | < .001 | 2.25 |
| Random Forest vs. SVM | 0.042 | [0.027, 0.057] | < .001 | 1.49 |
| SVM vs. Logistic Regression | 0.021 | [0.007, 0.036] | .001 | 0.76 |

¹ d computed as mean difference divided by the pooled residual SD ($\sqrt{\text{MSE}} = .0281$); not part of the original output, included for effect-size reporting.

6.1.2 Non-Technical Business Report

Four predictive algorithms were tested head-to-head on the same accuracy benchmark. The result is unambiguous: Gradient Boosting is the top performer, Random Forest is a clear second, Support Vector Machine is third, and Logistic Regression trails all three. The gap between the best and worst performer is approximately nine percentage points of accuracy, and statistical testing confirms this gap is real, not a product of random variation in the sample. All underlying data-quality checks passed, so the result can be relied upon as stated. The open question is not whether Gradient Boosting wins on accuracy — it does, decisively — but whether accuracy alone is the right basis for a deployment decision, given that cost, speed, and interpretability were not part of this comparison.

6.1.3 Recommendations

1. **Shortlist Gradient Boosting** as the lead candidate for further development, on the strength of a statistically validated and practically meaningful accuracy advantage over

all alternatives.

2. **Do not finalize the deployment decision on accuracy alone.** Gather comparative data on training cost, inference latency, and interpretability before committing, since these factors were outside the scope of this analysis and may offset the accuracy gain depending on the application.
3. **Confirm comparison fairness retrospectively:** verify that all four algorithms received equivalent tuning effort and identical data splits. An accuracy advantage attributable to unequal tuning budgets would not be a genuine algorithm effect.
4. **Replicate on a held-out or temporally distinct dataset** before treating this ranking as generalizable, since a single benchmark run cannot rule out dataset-specific idiosyncrasy.
5. **Document the data-generating process** behind the 200 accuracy observations (e.g., repeated k-fold cross-validation, repeated random splits) — this directly affects whether the independence assumption underlying the reported p -values holds.

6.2 Two-Way Analysis of Variance

6.2.1 Technical Academic Report (Based on APA Guidelines of Reporting Results)

A two-way analysis of variance was conducted to examine the effects of Algorithm (Logistic Regression, Random Forest, Support Vector Machine, Gradient Boosting), Dataset (10 distinct datasets), and their interaction on classification accuracy.

Assumption checks supported the validity of the model. Levene’s test indicated homogeneity of variance across all 40 algorithm-by-dataset cells, $F(39, 160) = 0.91$, $p = .629$. The Shapiro-Wilk test indicated that residuals did not depart from normality, $W = 0.99$, $p = .587$. A Durbin-Watson test found no evidence of positive autocorrelation, $DW = 2.11$, $p = .786$.

| Source | df | SS | MS | F | p | η^2 |
|----------------------------|------|-------|------|-------|--------|----------|
| Algorithm | 3 | 0.25 | 0.08 | 99.18 | < .001 | .62 |
| Dataset | 9 | 0.01 | 0.00 | 0.81 | .612 | .02 |
| Algorithm $\times$ Dataset | 27 | 0.02 | 0.00 | 0.68 | .881 | .04 |
| Residuals | 160 | 0.133 | 0.00 | — | — | .33 |

The main effect of Algorithm was statistically significant and large, $F(3, 160) = 99.18$, $p < .001$, $\eta^2 = .616$. The main effect of Dataset was not significant, $F(9, 160) = 0.81$, $p = .612$, $\eta^2 = .02$, indicating that accuracy did not differ systematically across the ten datasets when averaged over algorithm. The Algorithm $\times$ Dataset interaction was not significant, $F(27, 160)$

$= 0.68$, $p = .881$, $\eta^2 = .04$, indicating that the relative ordering of the four algorithms did not vary meaningfully from one dataset to another.

Post hoc comparisons: Algorithm. Tukey’s HSD confirmed that all six pairwise algorithm differences were statistically significant, reproducing the strict ordering Gradient Boosting > Random Forest > Support Vector Machine > Logistic Regression.

| Comparison | Diff. | 95% CI | p (adj.) | d^1 |
|---|-------|--------------|------------|-------|
| Gradient Boosting – Logistic Regression | 0.09 | [0.08, 0.11] | < .001 | 3.12 |
| Gradient Boosting – SVM | 0.07 | [0.05, 0.08] | < .001 | 2.38 |
| Gradient Boosting – Random Forest | 0.03 | [0.01, 0.04] | < .001 | 0.93 |
| Random Forest – Logistic Regression | 0.06 | [0.05, 0.08] | < .001 | 2.20 |
| Random Forest – SVM | 0.04 | [0.03, 0.06] | < .001 | 1.46 |
| SVM – Logistic Regression | 0.02 | [0.01, 0.04] | .002 | 0.74 |

¹ d = mean difference / $\sqrt{\text{MSE}}$ ($\sqrt{0.00083} = 0.0288$); included for effect-size context, not part of original output.

Post hoc comparisons: Dataset. Consistent with the non-significant main effect, none of the 45 pairwise dataset comparisons reached significance (p adj. range = .838–1.000).

Post hoc comparisons: Algorithm $\times$ Dataset. Given the non-significant omnibus interaction, the 780 individual cell-level contrasts are not interpreted or reported in full. A number of these contrasts nonetheless show small adjusted p -values (e.g., Gradient Boosting:Dataset_1 vs. Logistic Regression:Dataset_2, $p = .0002$); this pattern reflects the large main effect of Algorithm surfacing within individual cell pairs, not a true interaction, and should not be read as evidence that algorithm performance depends on dataset. Standard practice is to withhold interpretation of simple effects when the omnibus interaction test is non-significant, and that practice is followed here.

6.2.2 Non-Technical Business Report

The earlier finding — that Gradient Boosting outperforms the other three algorithms — has now been stress-tested across ten distinct datasets, and it holds without exception. Performance does not vary meaningfully by dataset on its own, and, critically, the ranking of algorithms (Gradient Boosting first, then Random Forest, then SVM, then Logistic Regression) is identical across all ten datasets — there is no scenario in this evidence base where a different algorithm comes out on top. This substantially strengthens the earlier conclusion: the advantage is not a quirk of one dataset or one lucky sample, but a consistent pattern. All data-quality checks passed. The remaining decision is the same one flagged previously: accuracy is settled, but cost, latency, and interpretability still need to be considered before committing to a production choice.

6.2.3 Recommendations

1. **Proceed with Gradient Boosting as the lead candidate** with higher confidence than before — the result is now supported by consistency across ten datasets, not a single benchmark run.
2. **Establish precisely what the ten “datasets” represent** before treating this as evidence of cross-domain generalization. This single clarification materially changes how strong a claim can be made.
3. **Still price in cost, latency, and interpretability** before finalizing deployment, as these remain untested dimensions.
4. **Do not interpret the individual Algorithm $\times$ Dataset cell comparisons** in any downstream reporting; only the omnibus interaction test (non-significant) is the supportable claim.
5. **Validate on at least one genuinely held-out dataset** collected independently of the original ten, to confirm the result is not an artifact of how those ten datasets were originally selected or constructed.

6.3 Multiple Analysis of Variance

6.3.1 Technical Academic Report (Based on APA Guidelines of Reporting Results)

A one-way multivariate analysis of variance (MANOVA) was conducted to examine the effect of Algorithm (Logistic Regression, Random Forest, Support Vector Machine, Gradient Boosting) on a combined outcome of classification Accuracy, Precision, and Recall.

Assumption checks. Mardia’s test indicated no significant departure from multivariate normality in skewness, $b = 10.71$, $p = .381$, or kurtosis, $b = 0.28$, $p = .782$. Box’s M test indicated no significant violation of the homogeneity of covariance matrices assumption, $^2(18) = 12.36$, $p = .828$. A Durbin-Watson test found no evidence of positive autocorrelation, $DW = 2.16$, $p = .815$.

Multivariate effect. The multivariate effect of Algorithm on the combined outcome was statistically significant, Pillai’s Trace = 0.67, $F(9, 588) = 18.60$, $p < .001$.

Univariate follow-up tests. Given the significant multivariate effect, univariate ANOVAs were examined for each outcome as protected follow-up tests.

| Outcome | <i>df</i> | <i>MS</i> | <i>F</i> | <i>p</i> | ² |
|-----------|-----------|-----------|----------|----------|--------------|
| Accuracy | 3, 196 | 0.0823 | 104.75 | < .001 | .616 |
| Precision | 3, 196 | 0.0833 | 74.55 | < .001 | .533 |

| | | | | | |
|--------|--------|--------|-------|--------|------|
| Recall | 3, 196 | 0.0838 | 77.71 | < .001 | .543 |
|--------|--------|--------|-------|--------|------|

All three effects were significant and large, with Algorithm explaining between 53% and 62% of variance depending on outcome.

Post hoc comparisons. Tukey's HSD was conducted separately for each outcome.

Accuracy

| Comparison | Diff. | 95% CI | <i>p</i> (adj.) | <i>d</i> ¹ |
|---|-------|--------------|-----------------|-----------------------|
| Gradient Boosting – Logistic Regression | 0.09 | [0.08, 0.11] | < .001 | 3.21 |
| Gradient Boosting – SVM | 0.07 | [0.05, 0.08] | < .001 | 2.45 |
| Gradient Boosting – Random Forest | 0.03 | [0.01, 0.04] | < .001 | 0.95 |
| Random Forest – Logistic Regression | 0.06 | [0.05, 0.08] | < .001 | 2.26 |
| Random Forest – SVM | 0.04 | [0.03, 0.06] | < .001 | 1.50 |
| SVM – Logistic Regression | 0.02 | [0.01, 0.04] | .001 | 0.76 |

Precision

| Comparison | Diff. | 95% CI | <i>p</i> (adj.) | <i>d</i> ¹ |
|---|-------|--------------|-----------------|-----------------------|
| Gradient Boosting – Logistic Regression | 0.09 | [0.07, 0.11] | < .001 | 2.64 |
| Gradient Boosting – SVM | 0.07 | [0.05, 0.08] | < .001 | 2.00 |
| Gradient Boosting – Random Forest | 0.02 | [0.00, 0.04] | .018 | 0.59 |
| Random Forest – Logistic Regression | 0.07 | [0.05, 0.09] | < .001 | 2.05 |
| Random Forest – SVM | 0.05 | [0.03, 0.06] | < .001 | 1.40 |
| SVM – Logistic Regression | 0.02 | [0.00, 0.04] | .008 | 0.65 |

Recall

| Comparison | Diff. | 95% CI | <i>p</i> (adj.) | <i>d</i> ¹ |
|---|-------|--------------|-----------------|-----------------------|
| Gradient Boosting – Logistic Regression | 0.09 | [0.07, 0.11] | < .001 | 2.69 |
| Gradient Boosting – SVM | 0.07 | [0.05, 0.08] | < .001 | 1.99 |
| Gradient Boosting – Random Forest | 0.02 | [0.00, 0.04] | .030 | 0.56 |
| Random Forest – Logistic Regression | 0.07 | [0.05, 0.09] | < .001 | 2.14 |
| Random Forest – SVM | 0.05 | [0.03, 0.06] | < .001 | 1.43 |
| SVM – Logistic Regression | 0.02 | [0.01, 0.04] | .003 | 0.71 |

¹*d* = mean difference / $\sqrt{\text{MSE}}$; not part of the original output, included for effect-size context.

Synthesis. Across all three outcomes, every pairwise comparison was statistically significant, and the rank ordering was identical: Gradient Boosting > Random Forest > SVM > Logistic Regression. This convergence across three distinct, though related, performance metrics provides stronger evidence for the algorithm ranking than any single outcome could provide alone. The Gradient Boosting–Random Forest contrast was consistently the smallest margin among the six comparisons in every outcome and is identified below as the comparison most sensitive to multiplicity correction.

6.3.2 Non-Technical Business Report

The conclusion is now fully confirmed: Gradient Boosting leads, Random Forest is second, SVM third, Logistic Regression last — and this exact ordering holds independently across all three quality measures (accuracy, precision, and recall), with every single pairwise difference statistically significant in every measure. This is the most complete evidence produced across this evaluation: no algorithm wins on one metric while losing on another, and no result depends on which single metric is chosen to judge the algorithms. The only nuance worth flagging at the leadership level is that the gap between the top two performers — Gradient Boosting and Random Forest — is consistently the narrowest of the six comparisons across all three metrics. The overall ranking is not in question; how decisively Gradient Boosting separates itself from the runner-up is the one place where the margin is closer than the headline result suggests.

6.3.3 Recommendations

1. **Proceed with Gradient Boosting as the recommended algorithm.** The conclusion is now supported by full convergence across three independent performance metrics, the strongest evidentiary standard reached in this evaluation.
2. **Treat the Gradient Boosting vs. Random Forest margin as the genuine decision point**, not the broader ranking. If training cost, inference latency, or interpretability favor Random Forest, this is the one comparison where that trade-off is close enough to matter.
3. **Apply a stricter correction across the three outcome families** (e.g., Bonferroni at $= .0167$ instead of $.05$) specifically to the Gradient Boosting–Random Forest contrasts before treating them as fully secure, since Tukey’s correction here only protects within each individual metric’s six comparisons, not across all eighteen.
4. **Report all three Tukey tables together**, as done above, rather than any one in isolation — the convergence across metrics is itself part of the evidentiary strength and should not be dropped in summarized reporting.

5. **If forced to choose between Gradient Boosting and Random Forest under resource constraints**, treat it as a genuine cost-benefit decision rather than a statistical formality, given how narrow and comparatively fragile that specific margin is.

7 References and Further Reading

American Psychological Association. (2025, February). *Journal Article Reporting Standards (JARS)*. APA Style. Retrieved April 28, 2025, from <https://apastyle.apa.org/jars>

Hodeghatta, U. R., & Nayak, U. (2023). *Practical Business Analytics Using R and Python: Solve Business Problems Using a Data-driven Approach* (2nd ed.). Apress. <https://link.springer.com/book/10.1007/978-1-4842-8754-5>